



江苏海洋大学
JIANGSU OCEAN UNIVERSITY

本科毕业设计 (论文)

一种支持并发执行的脚本编程语言设计与编译器实现
**Design and Implementation of a Concurrent Tcl-style
CPU/CUDA Hybrid Computational Geometry Scripting
System**

学 院: 计算机工程学院

专 业 班 级: 软嵌 1222

学 生 姓 名: 房鸿铭 学号: 2022122407

指 导 教 师: 纪兆辉 (副教授)

2026 年 5 月 28 日

江苏海洋大学

毕业设计（论文）原创性声明

本人声明所提交的毕业设计（论文）是在导师的指导下，独立完成的研究成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或者集体已经发表或者撰写过的成果。本文的研究成果是经过本人的辛勤劳动取得的，对本研究做出重要贡献的个人和集体，均已在文中明确表示出来。本人完全意识到本声明所涉及的法律后果由本人承担。

作者签名：房江鸣

日期：2026 年 5 月 28 日

江苏海洋大学

毕业设计（论文）版权使用授权书

本毕业设计（论文）作者同意学校保存并送交国家有关部门或机构有关毕业设计（论文）的复印件、电子版，允许论文被查阅和借阅。本人同意江苏海洋大学可以将本毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或者扫描等复制手段保存和汇编本毕业设计（论文）。

作者签名：房江鸣

指导教师签名：纪兆辉

日期：2026 年 5 月 28 日

日期：2026 年 5 月 28 日

毕业设计（论文）中文摘要

一种支持并发执行的脚本编程语言设计与编译器实现

摘 要：在海洋牧场监测、智慧港口安全、无人机巡检和船舶碰撞检测等场景中，常常需要处理大量点、区域、距离和碰撞关系。这类任务用脚本组织流程比较方便，但普通脚本程序很难直接利用 GPU 的并行计算能力；如果直接编写 CUDA 程序，虽然性能较高，却需要开发者手动管理设备内存、指针和同步，使用门槛较高。针对这一问题，本文设计并实现了 FTCL，一套 Tcl 风格的脚本系统，使用户可以用脚本描述几何计算任务，同时由系统负责 CPU 与 GPU 之间的数据管理和后端执行。

本文的核心工作是设计跨设备向量 UVec。UVec 将同一份逻辑数据组织在 CPU 和 CUDA 设备上，并自动记录哪一份数据是最新的。当 CPU 或 GPU 需要访问数据时，UVec 会按需完成同步，从而减少用户手工编写内存拷贝代码的负担。基于 UVec，本文实现了面向脚本调用的几何命令库，支持距离矩阵、最近点查询、圆形范围计数、包围盒归约、仿射变换和 AABB 碰撞检测等批量几何任务。为了支持并发执行，系统还将 UVec 句柄管理从全局锁细化为按句柄加锁，减少多个独立任务之间不必要的等待。

本文从正确性、性能和应用三个方面对系统进行验证。语义回归测试、CPU/GPU 等价测试、随机差分测试和边界测试表明系统结果可靠；单 GPU 和八卡 A800 实验表明，输出较小、计算量较大的任务能够获得明显 GPU 加速，并在多 GPU 环境下接近线性弱扩展；而输出规模很大的距离矩阵会暴露输出路径瓶颈。并发实验进一步说明，解释器调度、锁粒度和 GPU 资源竞争会影响系统吞吐量和尾延迟。最后，本文将 FTCL 应用于海洋牧场空间感知、智慧港口安全分析、无人机水面巡检和船舶碰撞检测等示例场景。实验结果表明，FTCL 不只是一个几何函数集合，而是一套能够把脚本表达、跨设备数据管理、GPU 加速和可复现实验连接起来的异构空间计算原型系统。

关键字：脚本解释器；Tcl；计算几何；UVec；CUDA；多 GPU；海洋空间计算

Design and Implementation of a Concurrent Tcl-style CPU/CUDA Hybrid Computational Geometry Scripting System

Abstract: In scenarios such as marine ranch monitoring, smart port safety, UAV inspection, and ship collision checking, many tasks involve large numbers of points, regions, distances, and collision relations. Scripts are convenient for organizing such workflows, but ordinary scripting programs can hardly use GPU parallel computing directly. CUDA can provide high performance, but it also requires developers to manage device memory, pointers, and synchronization by hand. To address this gap, this thesis designs and implements FTCL, a Tcl-style scripting system that allows users to describe geometry computation tasks in scripts while the system manages data movement between CPU and GPU and dispatches the corresponding backend execution.

The core design of FTCL is UVec, a cross-device vector abstraction. UVec organizes the same logical data on both CPU and CUDA devices, records which copy is up to date, and performs synchronization on demand when the CPU or GPU needs to access the data. This reduces the burden of manually writing memory-copy code. Based on UVec, the thesis implements a script-facing geometry command library that supports batch geometry tasks such as distance matrices, nearest-point queries, circle range counting, bounding-box reduction, affine transformations, and AABB collision detection. To support concurrent execution, the system further refines UVec handle management from one global lock to per-handle locking, reducing unnecessary waiting among independent tasks.

The system is evaluated from correctness, performance, and application perspectives. Semantic regression tests, CPU/GPU equivalence tests, random differential tests, and edge-case tests show that the system produces reliable results. Single-GPU and eight-GPU A800 experiments show that workloads with small outputs and large computation can obtain clear GPU acceleration and achieve near-linear weak scaling in a multi-GPU environment, while distance matrices with large outputs expose output-path bottlenecks. Concurrency experiments further show that interpreter scheduling, lock granularity, and GPU resource contention affect throughput and tail latency. Finally, FTCL is applied to example scenarios including marine ranch spatial sensing, smart port safety analysis, UAV water-surface inspection, and ship collision checking. The results show that FTCL is not merely a collection of geometry functions, but a prototype heterogeneous spatial-computing system that connects scripting, cross-device data management, GPU acceleration, and reproducible experiments.

Keywords: scripting interpreter; Tcl; computational geometry; UVec; CUDA; multi-GPU; marine spatial computing

目 录

1	引言	1
1.1	研究背景	1
1.2	研究动机	1
1.3	研究问题	2
1.4	主要贡献	2
1.5	论文组织	3
1.6	小结	4
2	相关工作与技术背景	5
2.1	Tcl 风格解释执行 (Tcl-style Interpretation)	5
2.2	解析器设计 (Parser Design)	5
2.3	表达式求值 (Expression Evaluation)	6
2.4	计算几何 (Computational Geometry)	6
2.5	CUDA 与端到端性能 (End-to-end Performance)	6
2.6	异构系统案例研究 (Heterogeneous System Case Studies)	7
2.7	可复现实验产物 (Reproducible Experiment Artifacts)	7
2.8	小结	7
3	系统设计	9
3.1	总体架构	9
3.2	工程规模	10
3.3	解释器运行时 (Interpreter Runtime)	10
3.4	词法分析流程 (Lexical Analysis Flow)	11
3.5	语法分析流程 (Syntax Analysis Flow)	13
3.6	语义执行流程 (Semantic Execution Flow)	16
3.7	UVec 跨设备设计	17
3.8	形式化 UVec 一致性模型	19
3.9	几何命令流程 (Geometry Command Flow)	21
3.10	并发设计	22
3.11	小结	22

4	系统实现	23
4.1	解释器核心 (Interpreter Core)	23
4.2	表达式解析器 (Expression Parser)	23
4.3	UVec 实现 (UVec Implementation)	25
4.4	CPU 几何算法 (CPU Geometry Algorithms)	27
4.5	CUDA 几何后端 (CUDA Geometry Backend)	27
4.6	脚本接口 (Script Interface)	28
4.7	小结	29
5	实验评估	30
5.1	实验目标 (Evaluation Goals)	30
5.2	实验方法 (Experimental Methodology)	30
5.2.1	CPU 基线定义 (CPU Baseline Definition)	31
5.3	语义回归测试 (Semantic Regression Tests)	32
5.4	解析器后端计时 (Parser Backend Timing)	33
5.5	并发与运行时平滑性 (Concurrency and Runtime Smoothness)	34
5.6	CPU/GPU 等价性验证 (CPU/GPU Equivalence)	36
5.7	几何性能扩展 (Geometry Performance Scaling)	36
5.8	多 GPU 扩展 (Multi-GPU Scaling)	38
5.8.1	多 GPU 弱扩展模型 (Weak-scaling Model)	40
5.9	盈亏平衡分析 (Break-even Analysis)	43
5.10	并发几何 (Concurrent Geometry)	44
5.11	小结	46
6	应用与讨论	47
6.1	从 STA 的 Tcl 范式到 FTCL 空间计算	47
6.2	FTCL 的空间计算 workflow	48
6.3	面向海洋的空间计算演示 (Spatial Computing Demo)	49
6.4	海洋牧场空间感知	50
6.5	智慧港口空间计算	51
6.6	无人机水面漂浮物巡检	51
6.7	船舶路径规划与碰撞检测	52
6.8	算法可视化	52

6.9 与普通几何库的区别	53
6.10 局限性	54
6.11 小结	54
结论与展望	56
致谢	57
参考文献	58
构建、测试与复现命令	61
.1 CPU 构建与测试	61
.2 CUDA 构建	61
.3 核心基准测试	62
.4 几何图表生成	62
.5 论文编译	62

第 1 章 引言

1.1 研究背景

脚本语言常被用作底层系统能力与上层应用逻辑之间的胶水层。Tcl 是这种设计思想的代表之一：脚本由命令组成，命令由若干个词组成，变量替换、命令替换和表达式求值都发生在运行时^[1,2]。这种**命令中心模型**（command-centered model）的优势在于语法浅、嵌入方便、扩展自然，新的能力往往可以通过增加一个 native command 完成，而不必重新设计整门语言。

计算几何（computational geometry）则是算法实验中非常适合展示工程价值的方向。点、线段、多边形、包围盒、距离、相交、最近点和范围查询等问题，广泛出现于程序设计竞赛、图形学、机器人、游戏、地图分析和空间计算任务中^[3,4]。其中大量工作负载具有天然并行性：距离矩阵的每个元素可以独立计算，范围查询的每个查询区域也可以独立处理。这使得计算几何非常适合作为 GPU 加速（GPU acceleration）的实验载体^[5]。

本文将这两条线结合起来，并放入海洋空间计算的应用语境中。FTCL 使用 Tcl 风格解释器作为控制层（control layer），以 C++ 作为宿主实现语言，以 CUDA 作为部分批量几何算法的加速后端（acceleration backend）。本文的目标不是简单实现一组几何函数，而是构建一个**可脚本化系统**（scriptable system）：使用者可以用脚本描述海洋牧场感知、智慧港口空间分析、无人机巡检或船舶碰撞检测，而无需直接面对 CUDA 内存、原始设备指针和同步这些底层细节。

1.2 研究动机

本项目的动机来自四个观察。

第一，解释器项目天然具有系统工程深度。它涉及解析、值表示、作用域、命令分发、错误处理、测试和性能评估。这些部分彼此牵连，任何一处设计都会影响语言可用性与后续扩展能力，因此它比单个算法实现更能体现完整工程能力。

第二，几何算法既有算法竞赛的亲近感，又容易被可视化验证。方向判定、线段相交这类基础操作足够小，可以说明算法正确性；距离矩阵、最近点、范围查询这类批量任务又足够大，可以展示 CPU/CUDA 与多 GPU 的性能差异。

第三，异构计算最难的不只是 kernel，而是数据抽象。如果每一个几何命令都要手工调用 `cudaMalloc`、`cudaMemcpy` 和 `cudaFree`，脚本层会变得难用，也难以测试。UVec 的作用正在于此：它提供一个类向量对象（vector-like object），同时维护 CPU 与 CUDA 物理副本，并按需完成同步。

第四，真实空间计算任务往往不是单设备任务。海洋监测或港口巡检可能同时包含大量传感器、检测目标、查询区域和独立任务。如果硬件平台提供多张 GPU，脚本系统就应当能够暴露这些设备，并把独立工作负载分发出去，而不是被解释器运行时的全局锁串行化。

1.3 研究问题

本文围绕以下研究问题展开。

RQ1: 一个紧凑的 Tcl 风格解释器能否为算法脚本、并发原语和扩展命令提供稳定的语义基础（semantic foundation）？

RQ2: 一个跨设备向量抽象能否隐藏 CPU/CUDA 数据搬运，同时保持接近 `std::vector` 的编程直觉？

RQ3: 当度量对象是端到端脚本命令延迟（end-to-end script command latency），而不是纯 kernel 时间时，哪些计算几何工作负载真正能从 CUDA 加速中受益？

RQ4: 哪些运行时开销会限制脚本驱动异构几何系统的加速效果，这些开销如何通过实验观察？

RQ5: 同一套脚本模型与 UVec 模型能否表达多 GPU 空间工作负载，不同几何算法的扩展瓶颈又在哪里？

这些问题把全文拆成五个层次：语言语义、内存抽象、算法加速、多 GPU 执行与实验分析。这样的分层很重要，因为脚本系统必须先正确、可用、可复现，性能数字才有意义。

1.4 主要贡献

本文的主要贡献如下。

1. **面向算法实验的脚本运行时**。FTCL 以 C++20 实现 Tcl 风格命令模型，支持变量替换、表达式求值、过程、集合、源码加载、测试命令和线程通道。
2. **跨设备向量抽象 UVec**。UVec 将一个逻辑向量映射到 CPU 与 CUDA 物理缓冲区，跟踪有效副本，按需同步，并提供统一的只读与可变原始指针接口。
3. **更细粒度的 UVec 句柄管理**。系统避免在执行长时间几何命令时持有全局 UVec 表锁。全局锁只保护句柄查找，单个 UVec 对象由按句柄条目锁保护，从而允许不同 CUDA 设备上的独立任务并发执行。
4. **脚本可调用的计算几何库**。geom 命令族将标量几何与批量几何暴露给 FTCL 脚本，使大规模点集和查询集可以通过 UVec 句柄传递，而不是反复序列化为脚本文本。
5. **批量几何算法的 CUDA 加速**。项目实现距离矩阵、最近点、范围计数、变换、方向判定和 AABB 碰撞等独立工作项算法的 CUDA 后端。
6. **多 GPU 扩展实验**。评估部分在八卡 A800 服务器上测量 1/2/4/8 GPU 弱扩展，说明紧凑输出的范围计数接近线性扩展，而距离矩阵受输出带宽限制。
7. **面向海洋的应用框架**。本文将点、圆、包围盒和距离矩阵映射到海洋牧场感知、智慧港口空间计算、无人机漂浮物巡检和船舶路径碰撞检测。
8. **可复现实验流水线**。项目包含语义测试、CPU/GPU 等价测试、随机差分测试、边界测试、性能扩展测试、消融研究、并发实验、多 GPU CSV 输出与论文图表。

本文的创新性不在于某一个孤立组件，而在于这些组件的整合。FTCL 不是单纯的解释器，UVec 不是单纯的容器，几何库也不是单纯的算法集合。三者结合后，形成了一个**脚本驱动的异构几何实验平台**（script-driven heterogeneous geometry experimentation platform）。

1.5 论文组织

第 2 章介绍相关技术与研究背景。第 3 章给出系统总体设计。第 4 章描述解释器、UVec 与几何后端的实现。第 5 章评估正确性与性能。第 6 章讨论应用场景、局限与改进方向。附录给出构建、测试、基准与论文编译命令。

1.6 小结

本章说明了本文为什么选择“脚本语言 + 计算几何 + 异构计算”这条路线。FTCL 的定位不是追求语法复杂度，而是用一个可控的命令模型承载算法实验、空间计算和 GPU 后端。UVec 则把最容易劝退使用者的设备内存问题收束为一个可解释、可测试、可扩展的数据抽象。

简言之，FTCL 将算法表达与设备级执行分离；本文的贡献正是围绕这一分离展开——脚本表达意图，UVec 保持数据驻留，CPU/CUDA 后端提供执行能力。

第 2 章 相关工作与技术背景

2.1 Tcl 风格解释执行 (Tcl-style Interpretation)

Tcl 风格语言的核心是**命令导向执行** (command-oriented execution)。一个命令的第一个词被解释为命令名 (command name)，其余词在完成替换 (substitution) 后作为参数 (arguments) 传入命令函数。这种模型与**以表达式为中心的语言** (expression-centered language) 不同：Tcl 类系统的语法层很浅，而真正的语义由命令提供。

FTCL 延续这一思想，但只实现项目所需的受控子集。它支持变量、数组、过程、控制流、列表、字典、表达式求值、源码加载和测试命令。本文并不试图实现完整 Tcl，而是把 FTCL 定位为**计算几何与异构计算实验** (heterogeneous-computing experiments) 的可控实验平台。

2.2 解析器设计 (Parser Design)

FTCL 当前包含两条**解析器路径** (parser path)。**传统解析器** (legacy parser) 直接读取字符流，常数开销小，适合作为稳定参考；**词法流解析器** (token-stream parser) 先进行词法分析 (lexical analysis)，再通过 `TokenCursor` 识别命令边界 (command boundary) 和词边界 (word boundary)。词法流路径并不必然更快，因为它额外存储 token 元数据，但更适合未来扩展基于 span 的诊断、AST 构建、字节码生成和解析器后端对比。

项目在迁移期间使用**影子对比** (shadow comparison)：两条解析器路径解析同一段输入并比较结构结果，以降低替换底层解析逻辑时的语义回归风险。

解析器与运行时设计 (parser and runtime design) 也与经典编译原理和解释器实现密切相关。编译原理教材讨论词法分析、语法分析、中间表示、优化与代码生成 [6,7]；解释器取向的参考更强调表示源程序结构、管理运行时值，并通过受控语义动作扩展语言 [8]。FTCL 当前选择后一条路线，同时保留 token-stream parser 作为未来进入 AST 与字节码执行的桥梁。

2.3 表达式求值 (Expression Evaluation)

FTCL 将**表达式求值** (expression evaluation) 与普通命令解析分离。`expr` 命令需要处理运算符优先级、结合性、短路逻辑、三目表达式、整数溢出、位运算、除法语义、变量引用和命令替换。FTCL 使用分层递归下降：低优先级函数调用高优先级函数，因此 `a || b && c` 会先在逻辑与 (logical AND) 层组合 `b && c`，再由逻辑或 (logical OR) 层处理整体表达式。

短路求值行为 (short-circuit behavior) 是表达式求值的关键。当 `||` 左侧为 `true` 时，右侧仍然需要被解析以保证解析位置正确，但不应产生副作用 (side effects)。FTCL 使用求值模式守卫 (evaluation mode guard) 实现这一点。

2.4 计算几何 (Computational Geometry)

计算几何研究点、线段、多边形和区域等几何对象上的算法。本文聚焦二维几何。基础操作包括点积、叉积、方向判定、距离、线段相交、点到线段距离、多边形面积、点在多边形内判定和凸包等。

批量几何算法 (batch geometry algorithms) 尤其适合 GPU 加速。距离矩阵对每个点对产生一个输出，范围计数查询对每个查询区域产生一个输出，最近点查询对每个查询点产生一个输出。这些任务包含大量**独立工作项** (independent work items)，因此可以自然映射到 CUDA 线程。

从算法基础看，本文与经典算法分析和数据结构设计一脉相承^[9,10,11]。范围计数、最近邻搜索和基于网格的加速等后续方向，则与多维数据结构密切相关^[12]。本文首先实现直接批量算法，是因为它们容易验证、易于并行，也适合做 CPU/GPU 等价性测试。

2.5 CUDA 与端到端性能 (End-to-end Performance)

CUDA 将 GPU 暴露为大规模并行设备 (massively parallel device)。典型 CUDA 程序需要分配设备内存、传输输入、启动 kernel、同步并把结果复制回主机 (host)。然而，脚本语言的使用者不应被迫手工管理这些步骤。FTCL 因此把这些底层行为封装在 `UVec` 与 `geom` 命令之后。

本文实验度量的是**端到端 FTCL 命令耗时** (end-to-end FTCL command time)，而不是纯 kernel 时间。一次几何命令测量包含命令分发、句柄查找、`UVec` 同步、

输出分配、kernel 启动、同步，有时还包含读回（readback）。这个口径更保守，但更接近脚本用户真实感受到的性能。

GPU 体系结构与 CUDA 系统论文为本文提供了重要参照。NVIDIA Tesla 提出了统一的图形与计算架构；CUDA 经验与优化研究则强调存储层次、线程组织和工作负载映射^[13,14,15,16]。OpenCL 与异构架构研究进一步说明，可移植性与数据搬运是 CPU/GPU 系统的核心问题^[18,17,19]。FTCL 的 UVec 层正是围绕这一点设计：后端代码可以拿到设备指针，但脚本不需要管理内存拷贝。

2.6 异构系统案例研究（Heterogeneous System Case Studies）

HeteroSTA 是一个具有代表性的 CPU/GPU 应用系统案例^[20]。它表明，异构引擎不仅需要并行 kernel，还必须处理调度、内存管理和工业 workflow 约束。FTCL 面向的是计算几何，而不是静态时序分析，但系统层面经验相似：加速效果只有在运行时、内存抽象、工作负载划分和基准方法一并设计时才有说服力。

2.7 可复现实验产物（Reproducible Experiment Artifacts）

本文将基准脚本、CSV 文件和 SVG/PNG 图表存放在 docs 目录下。这一点对于毕业论文非常重要，因为它把论文中的性能结论与可复现实验产物连接起来。本文的图不是手工绘制，而是由测试程序、基准数据和图表生成脚本自动生成。

为实现可复现研究，FTCL 的 C++ 实现源码、C++/Tcl 测试套件、几何与并发基准程序，以及论文图表生成脚本均已开源，托管于 GitHub 公共仓库 <https://github.com/homer-fang/ftcl>。读者可据此获取完整工程树，复现本文中的构建、测试、基准与图表流程；附录给出仓库地址及典型命令示例。

2.8 小结

本章说明 FTCL 所处的技术坐标：它借鉴 Tcl 的浅层语法与命令模型，采用解释器工程中的可控实现路线，并把 CUDA、UVec 与批量几何组合成一个实验平台。相关工作带来的核心启发是：GPU 加速不是单个 kernel 的胜利，而是语言运行时、数据搬运与工作负载结构共同作用的结果。

从实现路线看，FTCL 选择务实的解释器路径，而非完整编译器路径；其研究价值来自系统集成——小型命令语言、跨设备向量抽象，以及可复现的异构几何实验。

第 3 章 系统设计

3.1 总体架构

FTCL 的总体架构如图 3.1 所示。一个脚本首先被解析为命令（commands）和词（words），随后由解释器（interpreter）对词进行求值，完成变量替换、命令替换、数组访问和字符串拼接，最后将第一个求值结果作为命令名进行分发。`geom` 命令位于语言层和计算层之间：它负责解析几何参数或 UVec 句柄，并根据用户指定的设备（device）选择 CPU 后端或 CUDA 后端。UVec 则处在命令层与后端层之间，负责维护跨设备数据的位置、有效性和同步过程。

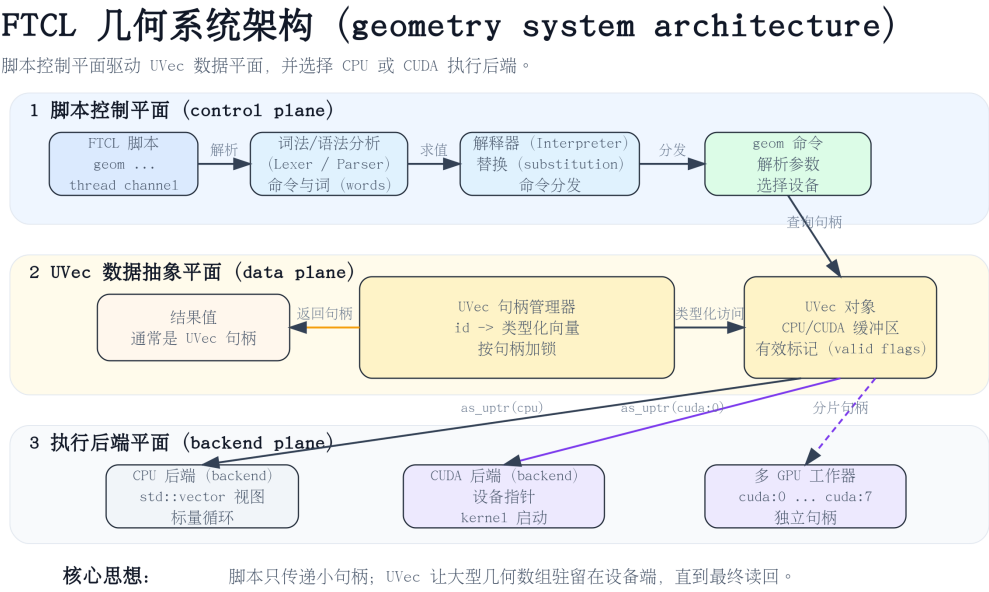


图 3.1: FTCL 几何脚本系统总体架构 (FTCL geometry scripting system architecture)

这一设计的核心是分层而不是堆叠。解析器与解释器专注于语言语义；命令层提供稳定的用户接口；UVec 负责数据驻留（data residency）与同步；CPU/CUDA 后端只实现具体算法。这样做的好处是：语言可以继续扩展，几何算法可以继续增加，CUDA 路径也可以在不破坏脚本接口的情况下逐步优化。

3.2 工程规模

表 3.1 总结了项目的主要工程组成。列出这张表的目的是不是简单展示文件数量，而是让系统边界变得清楚：FTCL 不是几个孤立的几何函数，而是由解释器、内存抽象、几何库、CUDA 后端、测试框架和基准流水线共同组成的完整原型系统。

表 3.1: FTCL 系统工程组成 (Engineering scope)

组件	主要文件	主要类/函数	作用
解释器核心	interp.hpp, type.hpp, parser.hpp	Interp, Value, Word, Script	负责脚本解析、求 值、变量、过程与命 令分发。
表达式解析器	expr.hpp	ExprParser, Datum	支持算术、比较、逻辑、 位运算、三目及 可替换表达式。
UVec 层	uvec.hpp, commands.hpp	UVec<T>, 指针包装器, 句柄管理器	提供跨设备向量存 储、CPU/CUDA 同 步与脚本可见句柄。
几何 CPU 后端	geometry.hpp	Point, Segment, BoundingBox, 批量算 法	在 CPU 数据上实现标 量与批量二维几何算 法。
CUDA 后端	geometry_cuda.*	kernel 与启动封装	为独立工作项几何算 法提供 GPU 加速。
测试	test/*.cpp	语义、等价、随机、 边界、压力测试	验证语言语义、UVec 一致性与 CPU/GPU 正确性。
基准与图表	docs/data, docs/scripts, docs/figures	CSV 与 Python 生成 器	生成可复现性能数据 与论文级图表。

3.3 解释器运行时 (Interpreter Runtime)

运行时 (runtime) 由值 (values)、词 (words)、脚本 (scripts)、作用域 (scopes) 和命令 (commands) 组成。Value 表示脚本层的值；Word 表示一个命令参数在求值前后的结构；Script 是命令列表；解释器维护命令表与变量作用域。FTCL 的执行路径保持 Tcl 风格的简单性，但在内部为后续扩展保留了结构化表示。

一个普通脚本的执行过程如下：

1. 将 source text 解析为 script structure。
2. 按顺序执行一个个 command。

表 3.2: 实现规模统计 (Quantitative implementation statistics)

指标	数值	含义
C++/CUDA 源文件数	56	<code>include</code> 、 <code>src</code> 和 <code>test</code> 下的头文件、源文件、CUDA、基准与测试文件。
C++/CUDA 源码行数	19151	<code>.hpp</code> 、 <code>.cpp</code> 和 <code>.cu</code> 文件总行数。
头文件/接口行数	12881	核心解释器、UVec、解析器、表达式、命令与几何接口。
CUDA/实现行数	642	CUDA 后端与非头文件源文件。
测试代码行数	5628	回归、等价、边界、随机、压力与基准相关测试代码。
CTest 条目	31	构建系统注册的回归与验证测试。
CUDA kernel 数	17	CUDA 几何后端及相关测试路径中的 device kernel。
基准可执行文件	6	用于运行时、几何与多 GPU 实验的非交互基准程序。
基准/图表脚本	5	<code>docs/scripts</code> 下的数据与图表生成脚本。
生成 SVG 图	18	<code>docs/figures</code> 下可复现矢量图。
论文 PNG 图	18	复制到 <code>contents/figures</code> 供论文编译使用。

- 3. 对每个 word 执行 variable substitution、command substitution、array lookup 和 string concatenation。
- 4. 将第一个求值后的 word 作为 command name。
- 5. 调用对应的 native C++ command function，并返回 value 或 exception。

这种结构有一个重要优势：新能力不需要发明新语法。`thread`、`uvec` 和 `geom` 都是 command，而不是语言核心的特殊分支。因此 FTCL 可以保持小而清晰，同时承载并发、跨设备内存和几何算法。

3.4 词法分析流程 (Lexical Analysis Flow)

词法层 (lexical layer) 将原始脚本字符串转换为 token 流 (token stream)。FTCL 的词法流方向会保留 token 文本与 token 边界，使后续阶段能明确识别命令结束符、注释、空白、花括号、方括号、变量引用和普通文本。算法 1 描述了词法分析器 (lexer) 的基本流程。

Algorithm 1: FTCL Lexical Analysis

Input: source string *source*
Output: token stream *tokens*

```

1 tokens  $\leftarrow$  [];
2 cursor  $\leftarrow$  0;
3 while cursor < |source| do
4   ch  $\leftarrow$  source[cursor];
5   if ch  $\in$  {space, tab} then
6     start  $\leftarrow$  cursor;
7     consume horizontal whitespace;
8     append Token(Whitespace, source[start : cursor]) to tokens;
9   else if ch is a newline or ch = ; then
10    append Token(CommandEnd, ch) to tokens;
11    cursor  $\leftarrow$  cursor + 1;
12  else if ch = # and cursor is at command start then
13    start  $\leftarrow$  cursor;
14    consume until newline or end of file;
15    append Token(Comment, source[start : cursor]) to tokens;
16  else if ch is a Tcl special delimiter then
17    append Token(Special, ch) to tokens;
18    cursor  $\leftarrow$  cursor + 1;
19  else
20    start  $\leftarrow$  cursor;
21    consume ordinary characters until a token boundary;
22    append Token(Text, source[start : cursor]) to tokens;
23 append Token(EndOfFile, empty string) to tokens;
24 return tokens;

```

Lexer 不执行 substitution。它只记录输入中可观察的结构。这样做非常重要，因为 Tcl-style substitution 本质上是 semantic action：\$ 可能需要 variable lookup，[...] 可能需要 recursive command evaluation。把 tokenization 和 evaluation 分开，可以让 parser 更稳定，也便于错误定位和回归测试。

3.5 语法分析流程 (Syntax Analysis Flow)

解析器 (parser) 消费 token 流, 并将 token 分组为命令与词。命令在换行、分号或文件结束处终止; 若解析器位于花括号、引号或方括号内部, 则这些符号不再表示命令边界。解析器不执行命令, 只构造后续可执行的脚本表示。

Algorithm 2: Token-stream Command Parsing

Input: token stream *tokens*

Output: script object *script*

1 **Procedure** ParseScript(*tokens*):

```

2   cursor ← TokenCursor(tokens);
3   script ← [ ];
4   while not cursor.atEnd() do
5       cursor.skipWhitespaceAndComments();
6       if cursor.atCommandEnd() then
7           cursor.next();
8           continue;
9       command ← ParseCommand(cursor);
10      append command to script.commands;
11      if cursor.atCommandEnd() then
12          cursor.next();
13      else if not cursor.atEnd() then
14          raise “expected command boundary”;
15  return script;
```

16 **Procedure** ParseCommand(*cursor*):

```

17   words ← [ ];
18   while not cursor.atEnd() and not cursor.atCommandEnd() do
19       cursor.skipInlineWhitespace();
20       if cursor.atCommandEnd() or cursor.atEnd() then
21           break;
22       word ← ParseWord(cursor);
23       append word to words;
24  return Command(words);
```

词法解析器（word parser）比命令解析器更细，因为一个 word 内部可能包含字面文本、变量引用、命令替换、数组引用、花括号字面量、引号字符串与展开语法。关键设计是：解析器将这些组成部分保存为结构化 word 节点，而解释器负责决定它们在运行时的值。

Algorithm 3: Word Parsing

Input: token cursor *cursor***Output:** word node**1 Procedure** ParseWord(*cursor*):

```

2   if cursor matches an open brace then
3       return Word.Value(ReadBalancedBracedText(cursor));
4   else if cursor matches a quote then
5       parts  $\leftarrow$  [];
6       while cursor does not match the closing quote do
7            $\lfloor$  append ParseWordPart(cursor, true) to parts;
8       return Word.Tokens(parts);
9   else if cursor matches the expansion prefix {*} then
10       child  $\leftarrow$  ParseWord(cursor);
11       return Word.Expand(child);
12   parts  $\leftarrow$  [];
13   while not cursor.atWordBoundary() do
14        $\lfloor$  append ParseWordPart(cursor, false) to parts;
15   if parts contains one literal part then
16       return Word.Value(parts[0]);
17   else
18        $\lfloor$  return Word.Tokens(parts);

```

19 Procedure ParseWordPart(*cursor*, *quoted*):

```

20   if cursor.peek() = $ then
21       return ParseVariableReference(cursor);
22   else if cursor.peek() = [ then
23       return Word.Script(ReadBalancedBracketScript(cursor));
24   else if cursor.peek() is a backslash then
25       return Word.String(ParseBackslashSequence(cursor));
26   else
27        $\lfloor$  return Word.String(ReadLiteralText(cursor, quoted));

```

3.6 语义执行流程 (Semantic Execution Flow)

语义执行 (semantic execution) 发生在语法分析之后。解释器逐条执行命令，对每个词求值，并把第一个值作为命令名，其余值作为参数。命令函数才真正改变变量、创建 UVec、启动 CUDA kernel 或返回结果。

Algorithm 4: Command-level Semantic Execution

Input: interpreter *interp*, parsed script *script*

Output: last command result

```

1 Procedure EvalScript(interp, script):
2   result  $\leftarrow$  empty string;
3   foreach command  $\in$  script.commands do
4     if command.words is empty then
5       continue;
6     values  $\leftarrow$  [];
7     foreach word  $\in$  command.words do
8       value  $\leftarrow$  EvalWord(interp, word);
9       if word is an expansion word then
10        extend values by ParseList(value);
11      else
12        append value to values;
13    name  $\leftarrow$  values[0].asString();
14    proc  $\leftarrow$  interp.lookupCommand(name);
15    if proc is missing then
16      raise "invalid command name";
17    result  $\leftarrow$  proc(interp, values);
18    if result.code is Return, Break, Continue, or Error then
19      propagate result according to the current context;
20  return result;

```

Algorithm 5: Word Evaluation**Input:** interpreter *interp*, word node *word***Output:** runtime value

```

1 Procedure EvalWord(interp, word):
2   if word.kind = Value then
3     return word.literal;
4   else if word.kind = String then
5     return word.text;
6   else if word.kind = VarRef then
7     return interp.lookupVariable(word.name);
8   else if word.kind = ArrayRef then
9     index  $\leftarrow$  EvalWord(interp, word.index);
10    return interp.lookupArrayElement(word.name, index);
11  else if word.kind = Script then
12    return EvalScript(interp, ParseScript(Tokenize(word.script)));
13  else
14    output  $\leftarrow$  empty string;
15    foreach part  $\in$  word.parts do
16      output  $\leftarrow$  output + EvalWord(interp, part).asString();
17  return output;

```

3.7 UVec 跨设备设计

UVec 是一个类向量容器 (vector-like container)，但它不只拥有 CPU 存储。普通 `std::vector<T>` 只维护 CPU 内存；UVec 同时维护 CPU 缓冲区与可选 CUDA 缓冲区，并记录哪一个设备上的数据是有效且最新的。

当 CPU code 请求最新数据，而最新副本在 CUDA 上时，UVec 执行 CUDA-to-CPU copy；当 CUDA kernel 请求最新数据，而最新副本在 CPU 上时，UVec 执行 CPU-to-CUDA copy；当某个 device 请求 mutable pointer 时，其他副本会被标记为 invalid，因为该 device 可能写入新值。图 3.2 展示同步机制，图 3.3 展示状态机视角。

UVec 跨设备同步 (cross-device synchronization)

读取会让目标设备副本变为有效；可写访问会让其他副本失效。

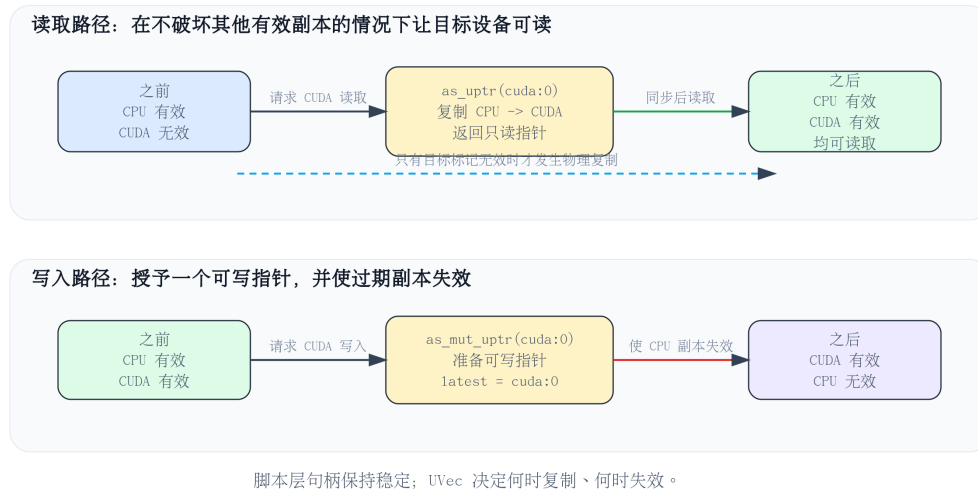


图 3.2: UVec 跨设备同步机制 (Cross-device synchronization in UVec)

UVec 状态机 (state machine)

读取可进入共享可读状态；可变写入会回到单一最新写入者。

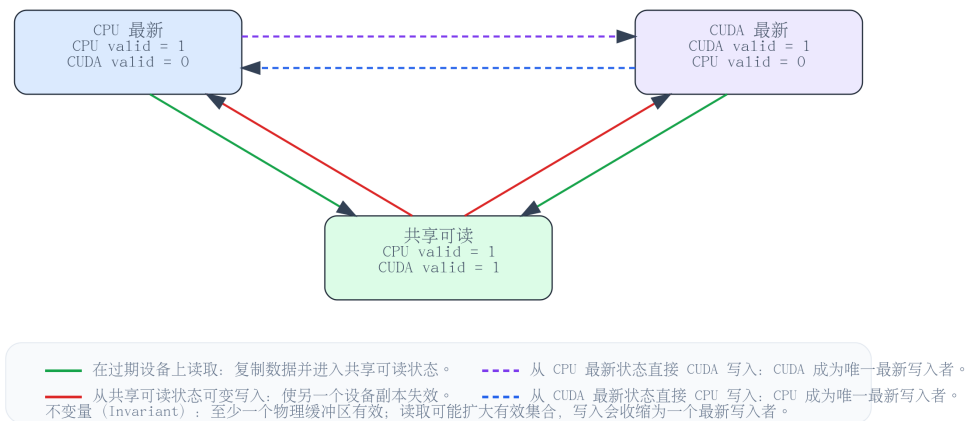


图 3.3: UVec 有效性状态机 (UVec validity-state machine)

最重要的两个 pointer interfaces 是 `as_uptr(device)` 和 `as_mut_uptr(device)`。前者返回 read-only pointer, 并保证目标 device 上的数据已经 valid; 后者返回 mutable pointer, 并将目标 device 标记为 newest writer。这两个接口让 CPU algorithm 和 CUDA kernel 可以共享同一个数据抽象, 而不需要脚本用户手动判断数据在哪里。

3.8 形式化 UVec 一致性模型

UVec 可以被描述为定义在设备集合上的小型**一致性模型**（consistency model）。令

$$D = \{\text{CPU}, \text{CUDA}_0, \text{CUDA}_1, \dots\}$$

表示可能的存储设备集合。长度为 n 的 UVec instance 表示为

$$U = (n, B, V, L),$$

其中 B_d 是 device $d \in D$ 上的 physical buffer, $V_d \in \{0, 1\}$ 表示 B_d 是否包含 valid copy, $L \subseteq D$ 表示可能包含 latest value 的 device set。当前实现通常表现为 single latest writer, 但集合表示允许 CPU 和 CUDA 在一次 read synchronization 后同时 valid。

两个核心操作满足以下规则:

Read rule: `as_uptr(d)` 只有在保证 $V_d = 1$ 后才能返回 device d 上的 pointer。如果 $V_d = 0$, 则从某个满足 $V_s = 1$ 的 source device s 复制数据, 并设置 $V_d \leftarrow 1$ 。Read operation 不会使其他 valid copies 失效。

Write rule: `as_mut_uptr(d)` 返回 device d 上的 mutable pointer。返回前必须保证 B_d 存在且包含 latest value; 随后设置 $V_d \leftarrow 1$, 并对所有 $s \neq d$ 设置 $V_s \leftarrow 0$, 因为通过 mutable pointer 的后续写入可能改变 logical vector。

在上述规则下, 可进一步形式化证明 UVec 维持有效副本的不变式。若 UVec 在初始化完成后满足

$$\exists d \in D, \quad V_d = 1,$$

且后续状态变化仅由 `as_uptr(d)` 与 `as_mut_uptr(d)` 两类操作产生, 则对任意有限操作序列, 下列两个性质成立:

$$\exists d \in D, \quad V_d = 1; \tag{3.1}$$

$$\text{as_uptr}(d) \text{ 成功返回时必有 } V_d = 1. \tag{3.2}$$

证明. 对操作序列长度 k 作数学归纳。这里主要证明有效副本不变式 (3.1); 对于读操作, 还需要同时说明性质 (3.2) 成立。

归纳奠基。当 $k = 0$ 时，系统尚未执行任何后续操作。由 UVec 的初始化条件可知，至少存在一个设备 $d \in D$ 满足 $V_d = 1$ ，因此不变式 (3.1) 成立。

归纳假设。假设执行前 k 次操作后，不变式 (3.1) 成立。也就是说，此时存在某个设备 $s \in D$ ，使得

$$V_s = 1.$$

归纳递推。考虑第 $k+1$ 次操作。根据 UVec 的接口定义，该操作只可能是 `as_uptr(d)` 或 `as_mut_uptr(d)`。

第一种情况：第 $k+1$ 次操作为 `as_uptr(d)`。

若操作前 $V_d = 1$ ，则设备 d 上已经有有效副本，系统可以直接返回该设备上的只读指针。此时至少设备 d 仍然有效，所以不变式 (3.1) 成立；同时，返回指针指向的正是设备 d 上的有效副本，因此性质 (3.2) 也成立。

若操作前 $V_d = 0$ ，则由归纳假设可知，当前系统中至少存在一个有效源设备 s ，满足 $V_s = 1$ 。根据 UVec 的读规则，系统会从该有效源设备同步数据到目标设备 d ，并设置

$$V_d \leftarrow 1.$$

读操作不会使其他有效副本失效，因此操作完成后至少设备 d 是有效的，从而仍有

$$\exists d \in D, \quad V_d = 1.$$

并且，由于 `as_uptr(d)` 在返回前已经保证 $V_d = 1$ ，所以返回的只读指针必然指向目标设备上的有效副本。

第二种情况：第 $k+1$ 次操作为 `as_mut_uptr(d)`。

根据 UVec 的写规则，系统在返回可变指针前，会先保证设备 d 上包含当前逻辑向量的最新数据。随后，设备 d 被标记为有效，其他设备上的旧副本被标记为无效，即

$$V_d \leftarrow 1, \quad V_s \leftarrow 0 \quad (s \neq d).$$

因此，即使其他设备上的副本被失效，设备 d 仍然保留一份有效副本。操作完成后仍满足

$$\exists d \in D, \quad V_d = 1.$$

这一分支对应的是可变指针访问，不涉及 `as_uptr(d)` 的返回条件，因此这里只需要维护有效副本不变式 (3.1)。

综上，如果前 k 次操作后有效副本不变式成立，那么第 $k + 1$ 次操作后该不变式仍然成立。由数学归纳法可知，对任意有限操作序列，UVec 始终至少保留一份有效副本。另一方面，在每次 $\text{as_uptr}(d)$ 成功返回前，系统都会保证目标设备 d 上的副本有效，因此性质 (3.2) 也成立。□

3.9 几何命令流程 (Geometry Command Flow)

geom 命令的执行流程如图 3.4 所示。以 `geom nearest_point` 为例，解释器首先求值脚本参数；命令层解析 UVec 句柄与设备选项；若目标是 CPU，则将输入同步到 CPU 并调用 C++ 算法；若目标是 CUDA，则将输入同步到 CUDA，创建输出 UVec，启动 CUDA kernel，并返回输出句柄。

geom 命令执行流程 (command execution flow)

示例: `geom nearest_point Sdataset Squries cuda:0`

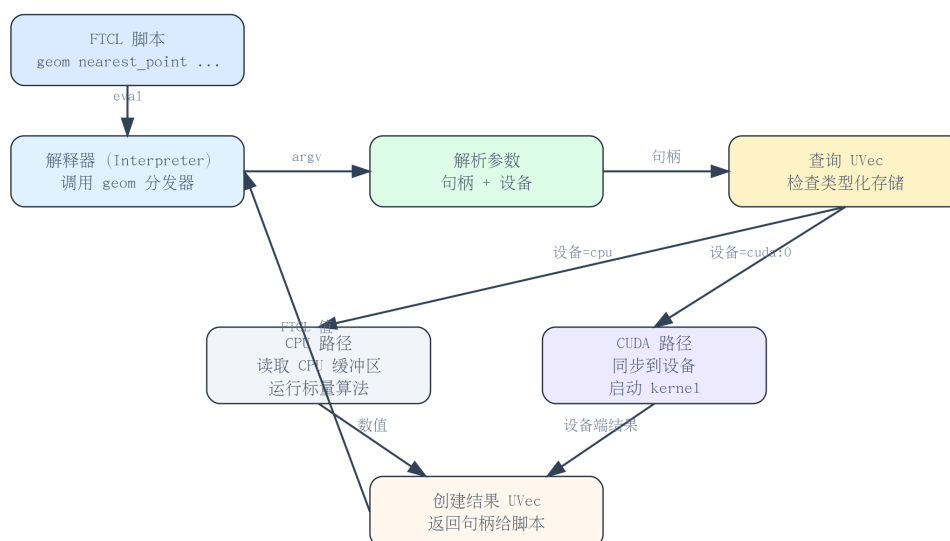


图 3.4: geom 命令执行流程 (Execution flow of a geometry command)

Handle 机制非常关键。大规模点集不应该在每条命令之间反复转换为 script strings。更合理的方式是: script 创建一次 UVec, 把 handle 传给多个 geometry commands, 只在最终需要展示或统计时再 read back。这个思路也是后面 performance evaluation 中 speedup 能出现的前提。

3.10 并发设计

FTCL 提供线程任务（thread tasks）与线程通道（thread channels）。线程任务可以异步运行一段脚本并在之后被等待（await）；线程通道是由互斥锁与条件变量保护的消息队列，支持阻塞接收与非阻塞尝试接收。

这个 concurrency model 对实验很有用。它可以驱动 game-like demo，模拟 producer-consumer workload，也可以通过多个 worker scripts 分发 geometry requests。当前 CUDA concurrency experiment 也暴露出一个限制：多个 script workers 可能竞争同一个 GPU，所以未来需要 streams 或 dedicated GPU scheduler。

后续第 5 章会分别从 single-GPU contention 和 multi-GPU weak-scaling 两个角度评价这个设计。这些实验也解释了为什么实现中需要把 UVec handle manager 从 global lock 细化为 handle-level locking。

3.11 小结

本章给出了 FTCL 的系统设计：语言层负责表达，命令层负责连接，UVec 层负责跨设备数据一致性，CPU/CUDA 后端负责具体计算。这一架构的关键不在于某一个单独模块，而在于模块边界足够清楚，从而能够支持后续测试、基准与应用场景扩展。

FTCL 将控制、数据驻留与执行后端分离；UVec 是连接脚本层几何命令与底层 CPU/CUDA 内存管理的桥梁。

第 4 章 系统实现

4.1 解释器核心 (Interpreter Core)

FTCL 的实现主要采用基于头文件 (header-based) 的结构, 核心代码位于 `include` 目录。解释器被构建为 CMake 接口库, 测试与基准可执行文件位于 `test` 目录。CUDA 支持由 `FTCL_ENABLE_CUDA` CMake 选项控制, 因此同一套代码可以在纯 CPU 环境与 CUDA 环境下分别构建。

值表示 (value representation) 遵循 Tcl 风格思想: 脚本层值首先是字符串, 但内部可以缓存结构化数据表示。例如, 一个值可以由整数、浮点数、布尔值、列表、字典或变量名构造; 当用户请求字符串表示时, 再延迟格式化 (lazy formatting)。这样既保持了面向字符串的脚本接口, 又减少了重复格式化带来的开销。

解释器核心的另一个设计目标是保持命令可扩展性。语言核心并不直接理解所有高级功能, 而是通过命令表分发。`expr`、`thread`、`uvec` 和 `geom` 都以命令形式进入系统。这使得 FTCL 可以逐步扩展, 而不需要频繁改变解析器文法。

4.2 表达式解析器 (Expression Parser)

表达式解析器 (expression parser) 使用递归下降优先级层次。顶层函数解析条件表达式, 然后逐层下降到逻辑或、逻辑与、位运算、相等性、关系运算、移位、加减、乘除、一元运算和基本表达式。

例如, `a || b && c` 由逻辑或层处理。左操作数先被解析为逻辑与表达式; 匹配到 OR 运算符后, 右操作数再次按逻辑与表达式解析, 因此 `b && c` 会先结合, 再参与 OR 运算。这种写法虽然不是 Pratt 解析器, 但优先级边界非常直观, 适合当前表达式子系统。

表达式解析器还支持短路求值 (short-circuit evaluation)。如果 `||` 的左操作数已经为 `true`, 右操作数仍会被解析器扫过以保持语法校验与解析位置, 但不会真正执行变量读取、命令替换或可能产生副作用的求值动作。这一点对 Tcl 风格表达式很重要, 因为表达式里可能包含 `$var` 和 `[script]`。

Algorithm 6: Expression Evaluation and Conditional Operator

Input: interpreter *interp*, expression string *input***Output:** expression value

```

1 Procedure EvalExpr(interp, input):
2   parser  $\leftarrow$  ExprParser(interp, input);
3   value  $\leftarrow$  parser.parseConditional();
4   parser.skipWhitespace();
5   if not parser.atEnd() then
6      $\mid$  raise “unexpected token in expression”;
7   return value;

8 Procedure ParseConditional(parser):
9   condition  $\leftarrow$  ParseLogicalOr(parser);
10  if not parser.match(?) then
11     $\mid$  return condition;
12  takeTrue  $\leftarrow$  condition.isTrue();
13  parser.setEvaluationEnabled(takeTrue);
14  trueValue  $\leftarrow$  ParseLogicalOr(parser);
15  parser.expect(:);
16  parser.setEvaluationEnabled( $\neg$ takeTrue);
17  falseValue  $\leftarrow$  ParseConditional(parser);
18  parser.restoreEvaluationMode();
19  if takeTrue then
20     $\mid$  return trueValue;
21  else
22     $\mid$  return falseValue;

```

Algorithm 7: Logical OR Short-circuit Evaluation**Input:** expression parser *parser***Output:** logical OR result

```

1 Procedure ParseLogicalOr(parser):
2   lhs  $\leftarrow$  ParseLogicalAnd(parser);
3   while parser.match(//) do
4     if lhs.isTrue() then
5       parser.setEvaluationEnabled(false);
6       ParseLogicalAnd(parser);           // parse only; no side effects
7       parser.restoreEvaluationMode();
8       lhs  $\leftarrow$  1;
9     else
10      rhs  $\leftarrow$  ParseLogicalAnd(parser);
11      lhs  $\leftarrow$  bool(lhs)  $\vee$  bool(rhs);
12  return lhs;

```

4.3 UVec 实现 (UVec Implementation)

UVec 要求元素类型满足 trivially copyable。这个约束不是形式主义，而是跨 CPU/CUDA 字节拷贝的基础：若类型需要复杂构造、析构或内部指针修正，则直接 memcpy 到设备内存并不安全。围绕 UVec 的主要辅助类型包括 Device、RawUPtr<T>、RawUMutPtr<T>、UniversalCopy 与 Zeroable。

脚本层 uvec 命令负责管理 UVec 句柄。它可以从列表创建向量，创建填充或零初始化向量，读取或设置元素，返回向量长度，同步到指定设备，并将向量转回脚本列表。命令管理器将句柄到 UVec 的映射放在 mutex 保护下，使脚本可以安全地在不同命令之间传递句柄。

Algorithm 8: UVec Read Synchronization

Input: UVec object *uvec*, target device *device***Output:** read-only unified pointer

```

1 Procedure AsUPtr(uvec, device):
2   if device = CPU then
3     if CPU buffer is not valid then
4       source  $\leftarrow$  any valid CUDA buffer;
5       copy source CUDA buffer to CPU buffer;
6       mark CPU buffer valid;
7     return RawUPtr(CPU pointer, length, CPU);
8   else
9     ensure CUDA buffer for device exists;
10    if target CUDA buffer is not valid then
11      if CPU buffer is valid then
12        copy CPU buffer to target CUDA buffer;
13      else
14        source  $\leftarrow$  another valid CUDA buffer;
15        copy source CUDA buffer to target CUDA buffer;
16      mark target CUDA buffer valid;
17    return RawUPtr(CUDA pointer, length, device);

```

Algorithm 9: UVec Mutable Pointer Access

Input: UVec object *uvec*, target device *device***Output:** mutable unified pointer

```

1 Procedure AsMutUPtr(uvec, device):
2   ensure storage exists on device;
3   if target device is not valid then
4     synchronize latest data to target device;
5   foreach other  $\in$  devices and other  $\neq$  device do
6     mark other invalid;
7   mark device valid and newest;
8   return RawUMutPtr(target pointer, length, device);

```

4.4 CPU 几何算法 (CPU Geometry Algorithms)

CPU 几何库实现在 `include/geometry.hpp` 中，基础类型包括点 (Point)、线段 (Segment) 和包围盒 (BoundingBox)。库中实现了距离、平方距离、点积、叉积、方向判定、点在线段上检测、线段相交、直线相交、点到直线距离、点到线段距离、多边形面积、多边形周长、点在多边形内判定、凸包、包围盒和最近点对距离。

批量算法使用扁平化数组存储。点集按 $x_0, y_0, x_1, y_1, \dots$ 存储；线段集按每条线段 x_1, y_1, x_2, y_2 存储；AABB 集合按 $min_x, min_y, max_x, max_y$ 存储。该格式对脚本、UVec 与 CUDA kernel 都友好，因为它避免了复杂对象布局，也便于跨设备拷贝。

表 4.1: 代表性几何算法及复杂度 (Representative geometry algorithms and asymptotic costs)

命令或函数	输入规模	CPU 复杂度	GPU 适用性
distance	一对点	$O(1)$	低；标量命令
segment_intersect	一对线段	$O(1)$	低；标量命令
polygon_area	P 个多边形顶点	$O(P)$	中等；适合批量多边形
convex_hull	N 个点	$O(N \log N)$	低；本项目未优先 GPU 化
bbox_reduce	N 个点	$O(N)$	高；适合并行归约
centroid	N 个点	$O(N)$	高；适合并行归约
batch_distance_matrix	N 乘 Q 个点	$O(NQ)$	很高；输出彼此独立
nearest_point	N 数据点, Q 查询	$O(NQ)$	高；查询彼此独立
range_count_circle	N 点, Q 圆	$O(NQ)$	高；查询彼此独立
batch_segment_intersect	K 线段对	$O(K)$	高；线段对彼此独立
collision_aabb	K 包围盒对	$O(K)$	高；包围盒对彼此独立
transform_points	N 个点	$O(N)$	很高；点彼此独立

表 4.1 解释了为什么并非每个几何命令都值得用 CUDA 加速。标量命令对脚本表达很有用，但计算量太小，难以摊销 kernel 启动开销。具有 $O(NQ)$ 或 $O(N)$ 个独立工作项的批量命令更适合 GPU。凸包这类算法需要排序与类栈的串行结构，因此本文将其保留为 CPU 优先算法。

4.5 CUDA 几何后端 (CUDA Geometry Backend)

CUDA 后端实现集中在 `src/geometry_cuda.cu`；其对外接口声明在 `include/geometry_cuda.hpp`。它支持距离矩阵、最近点、圆形范围计数、点变换、批量方向判定、AABB 碰撞、多边形批量面积与折线距离等数据并行算法。

以 N 个源点与 Q 个查询点构成的距离矩阵为例，输出规模为 NQ 。CUDA 映射非常直接：每个线程负责一个输出条目，读取一个源点与一个查询点，计算欧氏距离并写回结果。这类映射适合 GPU，因为每个输出元素彼此独立、控制分支少，且工作项数量可以快速增长。

4.6 脚本接口 (Script Interface)

下面的示例展示脚本如何创建两个点集，运行 CUDA 最近点查询，并读取结果。

Listing 4.1: FTCL geometry scripting example

```
set dataset [geom uvec_points {{0 0} {1 2} {3 4} {5 8}} cuda:0]
set query [geom uvec_points {{2 2} {4 6}} cuda:0]
set result [geom nearest_point $dataset $query cuda:0]
puts [uvec to_list $result]
```

用户不需要手动分配设备内存。脚本只描述任务；UVec 管理同步；geom 命令选择 CPU 或 CUDA 后端。这正是 FTCL 作为异构脚本系统的价值：把底层细节收进系统，把表达能力留给用户。

Algorithm 10: Geometry Command Dispatch for Nearest-point Query**Input:** interpreter *interp*, command arguments *argv***Output:** UVec handle containing nearest-point results

```

1 Procedure GeomNearestPoint(interp, argv):
2   datasetId  $\leftarrow$  ParseHandle(argv[2]);
3   queryId  $\leftarrow$  ParseHandle(argv[3]);
4   device  $\leftarrow$  ParseOptionalDevice(argv[4], CPU);
5   dataset  $\leftarrow$  uvecManager.lookup(datasetId);
6   queries  $\leftarrow$  uvecManager.lookup(queryId);
7   if device is CUDA then
8     datasetPtr  $\leftarrow$  dataset.asUPtr(device);
9     queryPtr  $\leftarrow$  queries.asUPtr(device);
10    output  $\leftarrow$  UVec.Uninitialized(queryCount  $\times$  2, device);
11    outputPtr  $\leftarrow$  output.asMutUPtr(device);
12    launch NearestPointCuda(outputPtr, datasetPtr, queryPtr);
13    return uvecManager.create(output);
14  else
15    datasetCpu  $\leftarrow$  dataset.cpuVector();
16    queriesCpu  $\leftarrow$  queries.cpuVector();
17    values  $\leftarrow$  NearestPointCpu(datasetCpu, queriesCpu);
18    output  $\leftarrow$  UVec.FromCpu(values);
19    return uvecManager.create(output);

```

4.7 小结

本章说明了 FTCL 从语言核心到 CUDA 后端的实现方式。解释器核心提供可扩展的命令分发；表达式解析器处理带短路求值的运行时表达式；UVec 把跨设备内存管理压缩为读写指针接口；几何库则把算法暴露为脚本命令。

实现刻意保持务实：优先清晰接口而非隐藏魔法。FTCL 之所以有用，是因为各层可以独立测试，并通过脚本可见命令组合起来。

第 5 章 实验评估

5.1 实验目标 (Evaluation Goals)

本章实验回答三个问题。第一，FTCL 的语言运行时是否保持了预期语义？第二，CPU 与 CUDA 几何后端是否能产生等价结果？第三，从 FTCL 脚本用户的角度看，CUDA 在什么规模下能够带来**端到端加速比** (end-to-end speedup)？

项目的实验数据、CSV 文件和图表生成脚本都位于 `docs` 目录下。测量内容包括语义通过率、解析器后端计时、线程通道延迟、帧时间分布、CPU/CUDA 几何加速比、吞吐量、数值误差、盈亏平衡点与并发几何性能。这些指标共同覆盖正确性、性能与可扩展性三个层面。

5.2 实验方法 (Experimental Methodology)

除非特别说明，本文所有性能结果都被视为**端到端系统测量** (end-to-end system measurements)。一次几何命令测量包含 FTCL 命令分发、参数解析、UVec 句柄查找、设备同步、输出 UVec 分配、后端执行，以及 CUDA kernel 启动后的同步。它不是纯 CUDA kernel time。本文有意采用这种测量方式，因为论文评价的是脚本用户实际看到的系统性能。

基准流水线 (benchmark pipeline) 遵循以下原则：

1. **使用 Release 构建**。Debug 构建不进入性能讨论，因为解释器分发与 STL 容器对优化级别非常敏感。
2. **区分预热与测量**。解析器后端计时与几何计时脚本在可支持的情况下会先执行预热运行，再记录样本。
3. **优先使用中位数或分位数**。延迟分布使用 P50、P95 和 P99 描述；几何计时在有多组样本时使用具有代表性的命令延迟中位数。
4. **单独看待读回成本**。部分实验将结果保留在 UVec 句柄中，避免把大规模输出转回脚本列表的成本混入主计时。
5. **先比较正确性，再解释速度**。只有 CPU/GPU 等价性测试在数值容忍范围内通过时，CUDA 加速比才有意义。

主要派生指标定义如下：

$$\text{speedup} = \frac{T_{\text{CPU}}}{T_{\text{CUDA}}}, \quad \text{throughput} = \frac{\text{work items}}{T}, \quad \text{work items} = N \times Q.$$

对于 distance matrix、nearest point 和 range-count 工作负载， N 表示数据点数量， Q 表示查询点或查询区域数量。

5.2.1 CPU 基线定义 (CPU Baseline Definition)

本文的 CPU 基线是 FTCL CPU 几何后端，而不是手工优化的并行 CPU 库。这个区别很重要：实验回答的是脚本用户会问的问题——同一个 FTCL 命令在 CPU 设备与 CUDA 设备上执行时，端到端提升是多少。

因此，CPU 行与 CUDA 行尽量共享相同的脚本层命令路径。主性能实验中，输入会在计时前预先构造为 UVec 句柄；计时区间包括命令分发、句柄查找、输出 UVec 分配、后端执行与必要同步。CPU 行使用 C++ 标量循环处理驻留在 CPU 的 UVec 数据；CUDA 行使用 CUDA 设备缓冲区与 kernel，且当前 CUDA 辅助函数在 kernel 启动后调用 `cudaDeviceSynchronize`，保证命令返回时后端工作已完成。

该基线对评价 FTCL 作为异构脚本系统是公平的，但不能被解释为对抗高度优化的全核 CPU 几何库。实验服务器拥有 128 个逻辑 CPU 线程，而当前 CPU 几何后端故意保持简单。因此，报告的加速比衡量的是：在相同命令接口下，将 FTCL 后端从标量 CPU 实现迁移到 CUDA 实现后获得的收益。未来加入全核 CPU 后端将是有价值的补充基线。

表 5.1: 实验环境信息 (Experimental environment for reproducibility)

Item	记录值
操作系统	用于构建、测试、CUDA 执行与图表生成的 WSL Ubuntu 环境
CPU 型号	Intel Xeon Platinum 8358, 2.60 GHz
CPU 拓扑	2 路插槽, 每路 32 核, 每核 2 硬件线程, 共 128 逻辑 CPU
CPU 频率	最低 800 MHz, 最高 3.40 GHz (lscpu 报告值)
NUMA 布局	2 个 NUMA 节点; 节点 0 含 CPU 0-31 与 64-95, 节点 1 含 CPU 32-63 与 96-127
主存	总内存 1.0 TiB; 记录环境时约 623 GiB 可用
GPU 型号	8 张 NVIDIA A800-SXM4-80GB
GPU 显存	每张 81920 MiB, 合计约 640 GiB 设备内存
GPU 驱动	NVIDIA driver 530.30.02
CUDA 计算能力	每张 A800 GPU 为 8.0
构建系统	CMake Release 配置
C++ 标准	C++20, 由 FTCL_CXX_STANDARD=20 指定
编译器	WSL 内可用的 GCC 或 Clang
CUDA 选项	FTCL_ENABLE_CUDA=ON, 用于 CUDA 几何实验
CPU 构建路径	build 或 /tmp/ftcl-build-geometry-cpu
CUDA 构建路径	/tmp/ftcl-build-geometry-cuda
基准数据	docs/data/benchmarks 与 docs/data/geometry 下的 CSV 文件
生成图表	docs/figures 与 contents/figures 下的 SVG/PNG 图
重要说明	可用 GPU 显存随时间变化, 因服务器可能被其他用户或作业共享

表 5.1 是环境记录, 而不是硬件无关结论。本文所有性能数据都应理解为在该多 GPU 服务器上的实测结果。服务器的 CPU 核心数也说明 CPU 基线并非来自弱硬件; 真正的实验边界在于当前 CPU 后端采用标量实现, 而非全并行 CPU 实现。

5.3 语义回归测试 (Semantic Regression Tests)

语义回归测试 (semantic regression tests) 覆盖核心命令、集合、控制流、表达式与线程。图 5.1 显示当前测量的测试套件达到 100% 通过率。

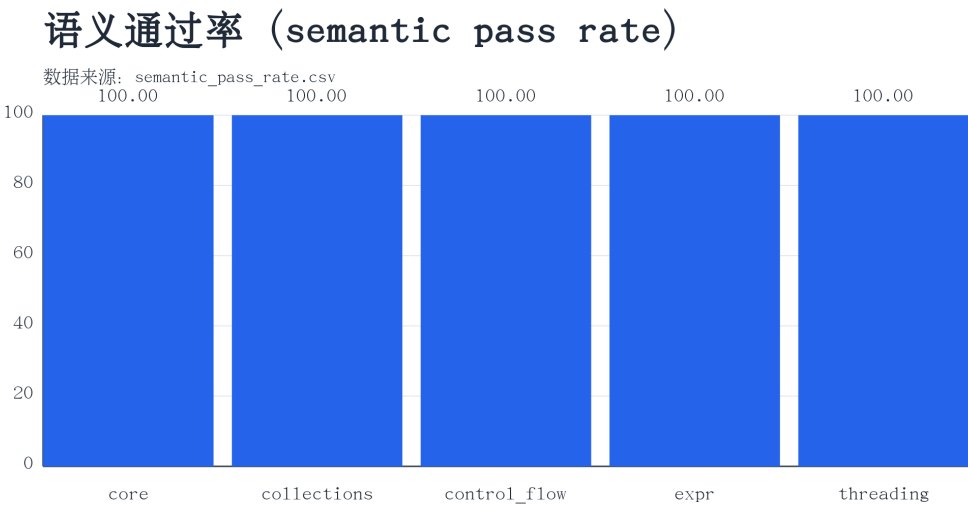


图 5.1: 测试套件语义通过率 (Semantic pass rate by test suite)

这个结果并不表示 FTCL 已经完整实现 Tcl。它表示本文选定的 Tcl subset 已经形成稳定 regression baseline。对于 parser 和 expression 这类容易引入细微语义变化的模块，这一点尤其重要。

5.4 解析器后端计时 (Parser Backend Timing)

图 5.2 比较传统解析器与词法流解析器在 Tcl 子集基准上的耗时。词法流路径并不一定更快，因为它增加了词法分析阶段并保存 token 元数据。其价值主要在架构层面：更好的诊断、更清晰的命令边界，以及通向 AST 与字节码执行的路径。

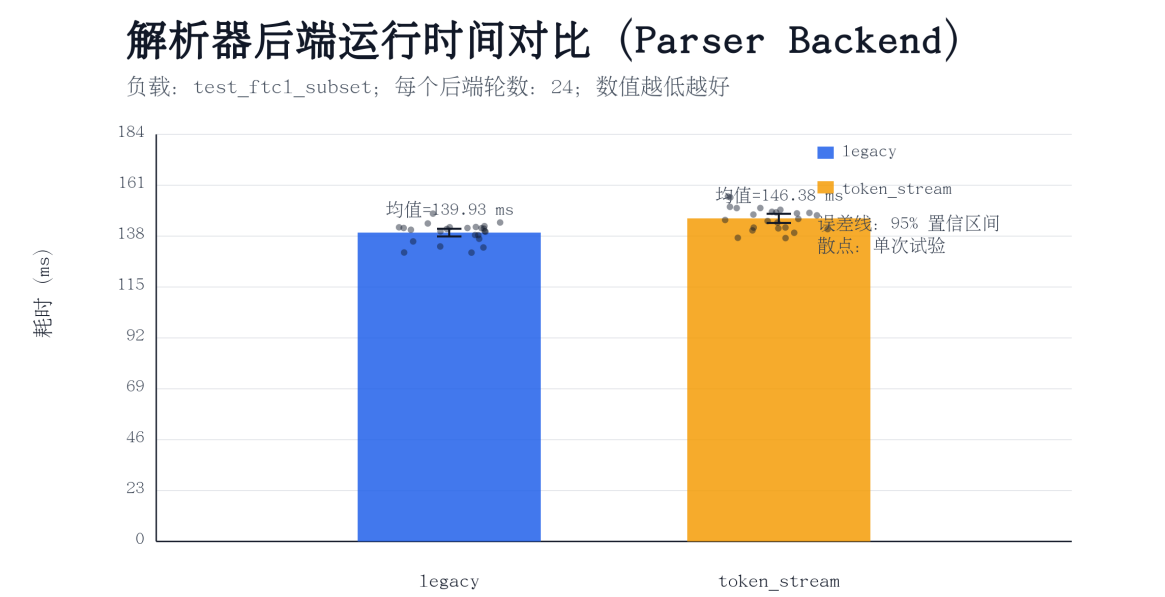


图 5.2: Parser backend 耗时对比 (Timing comparison between parser backends)

直接结论: token-stream 解析器的短期意义不是速度, 而是可维护性、错误定位与后续编译路径。

5.5 并发与运行时平滑性 (Concurrency and Runtime Smoothness)

线程通道延迟与帧时间分布如图 5.3 与图 5.4 所示。通道实验通过乒乓往返估计单向延迟; 帧时间实验测量一个非交互式类游戏的负载。

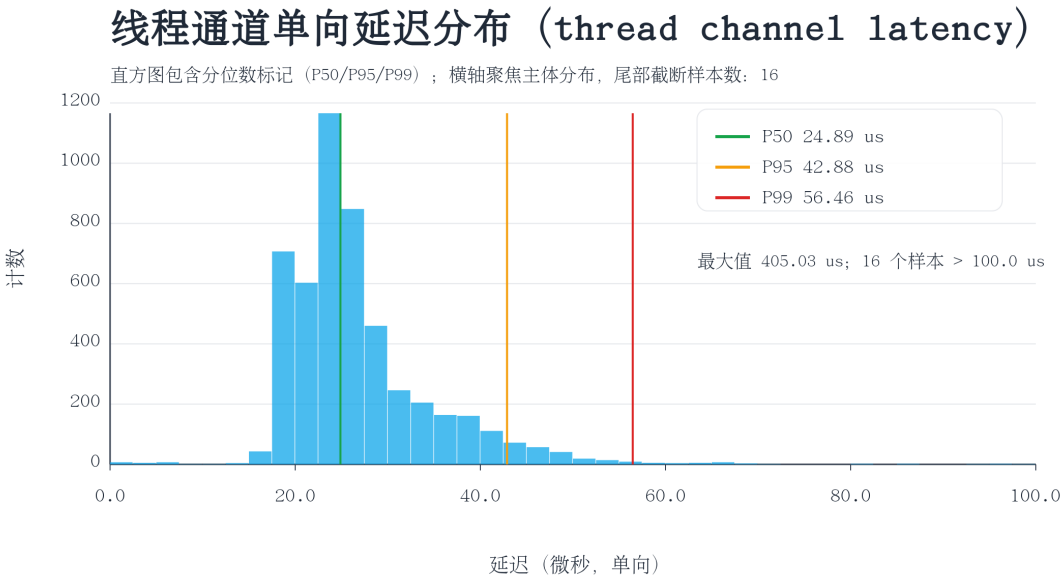


图 5.3: 线程通道单向延迟分布 (Thread-channel one-way latency distribution)

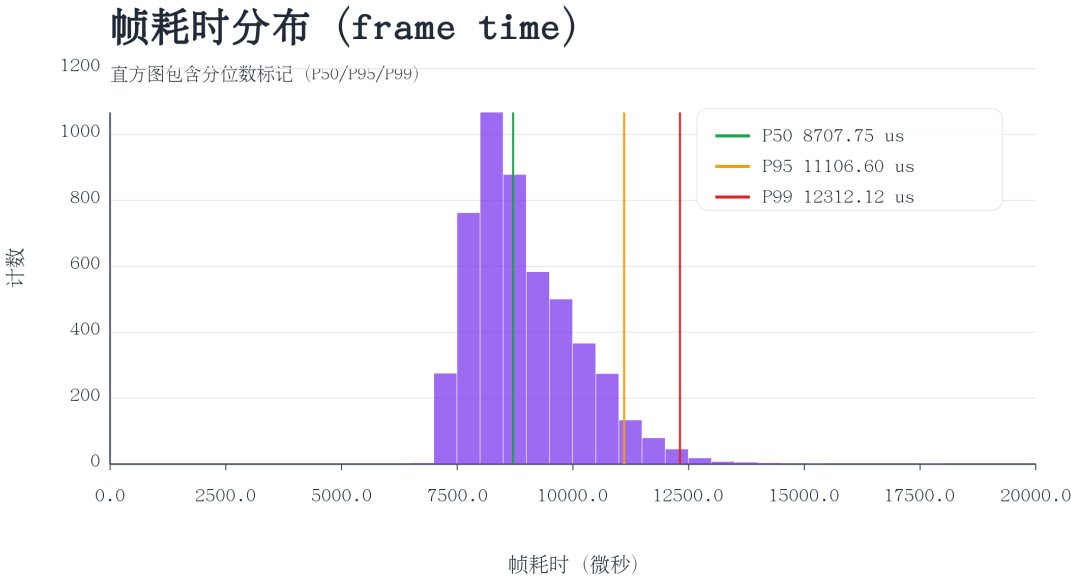


图 5.4: 类游戏 workload 的帧耗时分布 (Frame-time distribution)

这些实验说明 FTCL 可以支持简单的并发脚本与交互式演示。相比平均值, P50、P95 和 P99 等尾延迟指标更能反映脚本系统的实际交互体验。

5.6 CPU/GPU 等价性验证 (CPU/GPU Equivalence)

对 CUDA 后端而言，**正确性**比速度更重要。**等价性测试** (equivalence tests) 让同一几何工作负载同时在 CPU 与 CUDA 上运行，比较返回的 UVec 值，并允许较小的浮点容差。图 5.5 展示数值误差分布。

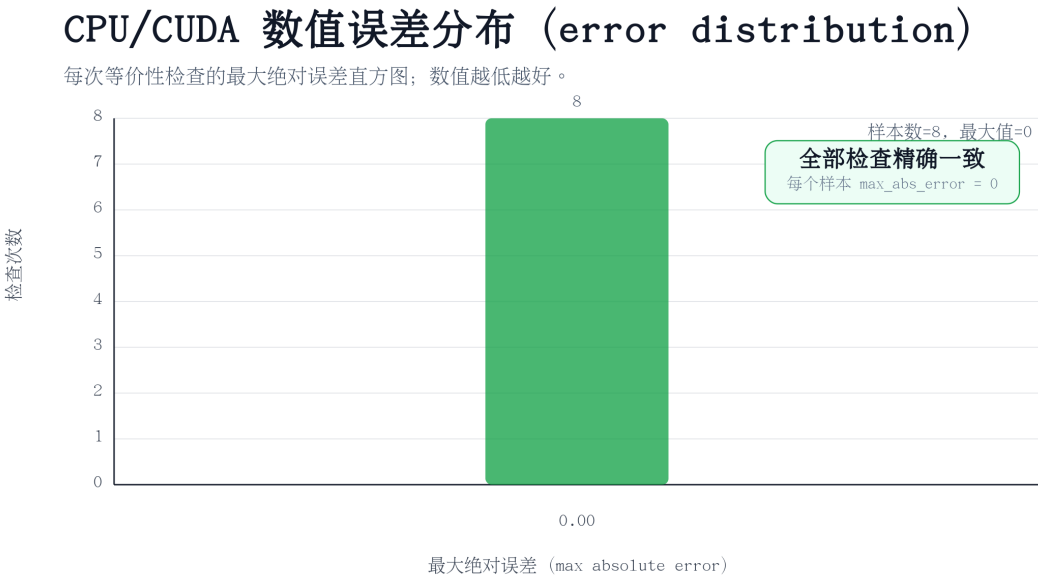


图 5.5: CPU 与 CUDA 几何结果的数值误差分布 (Numerical error distribution)

生成数据中的观测误差接近零。这支持了本文的基本正确性主张：在测试覆盖到的工作负载上，CUDA 实现保持了 CPU 几何语义。

5.7 几何性能扩展 (Geometry Performance Scaling)

性能实验聚焦三个代表性算法：距离矩阵、最近点与圆形范围计数。它们的共同点是存在大量独立工作项。表 5.2 根据保存的性能 CSV 总结端到端加速比。

单 GPU 性能扩展实验使用一个 FTCL 解释器和一个 CUDA 设备。每个规模下，基准会在计时前构造 CPU 与 CUDA UVec 输入，执行可选的非计时预热 (warmup)，并报告命令耗时的中位数。这意味着点列表解析成本被排除在主计时之外，而输出分配与后端同步仍包含在内。实验目标是测量预构建 UVec 句柄在 realistic 脚本流水线中的使用成本。

表 5.2: 代表性几何算法的端到端 CPU/CUDA 加速比

算法	N,Q	工作项	加速比
距离矩阵	2048, 256	524288	6.77x
最近点	2048, 256	524288	2.05x
圆形范围计数	2048, 256	524288	1.41x
距离矩阵	8192, 1024	8388608	51.90x
最近点	8192, 1024	8388608	8.39x
圆形范围计数	8192, 1024	8388608	6.50x
距离矩阵	32768, 2048	67108864	224.00x
最近点	32768, 2048	67108864	16.55x
圆形范围计数	32768, 2048	67108864	13.13x

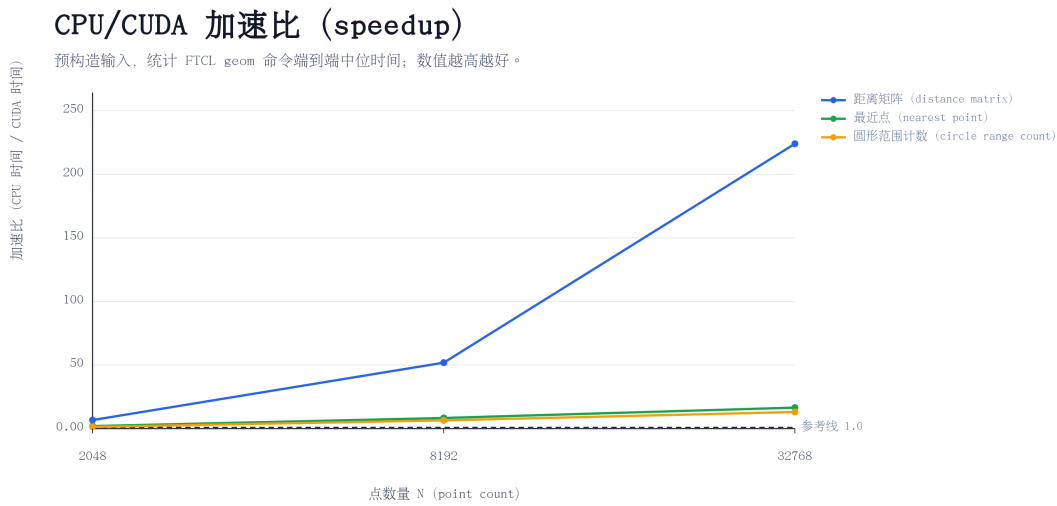


图 5.6: 批量几何算法的 CPU/CUDA speedup

图 5.7 与图 5.8 展示执行时间与吞吐量。距离矩阵获得最高加速比，因为其工作项简单且彼此独立。最近点与范围计数同样受益于 CUDA，但每个查询需要更多内存读取，因此扩展模式不完全相同。

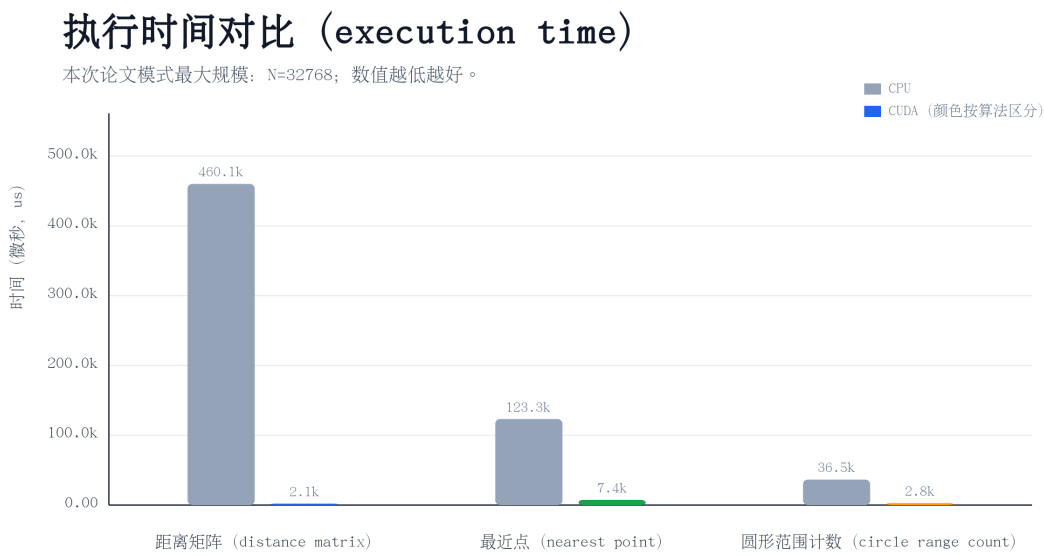


图 5.7: 最大测量规模下的执行时间对比 (Execution-time comparison)

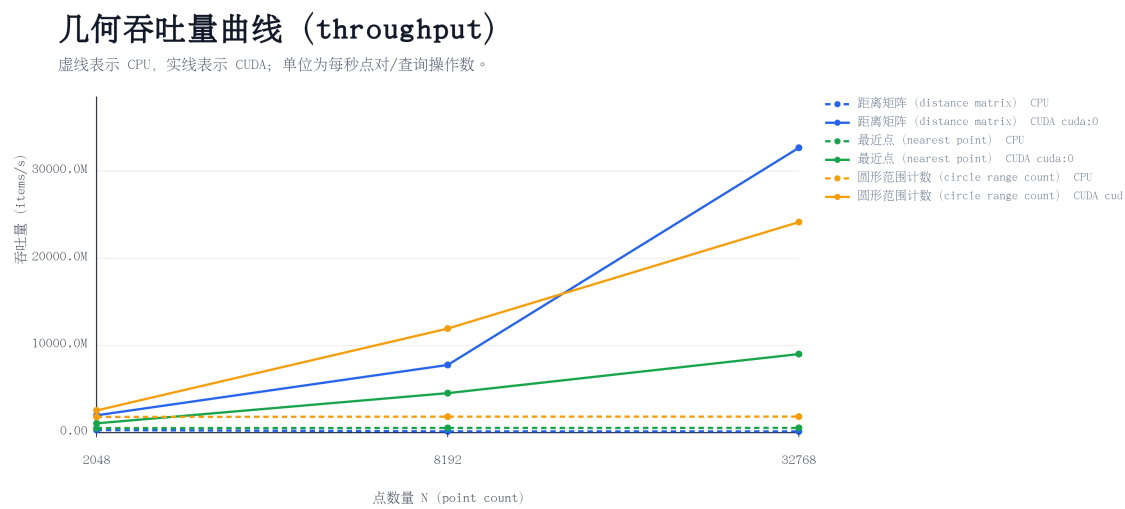


图 5.8: 几何算法吞吐量曲线 (Throughput curve)

直接结论: 工作项足够多且输出紧凑时, CUDA 后端的优势更稳定; 输出规模过大时, 分配与内存带宽会成为瓶颈。

5.8 多 GPU 扩展 (Multi-GPU Scaling)

实验服务器包含 8 张 A800 GPU, 因此本文进一步测量 FTCL 是否能把多个 CUDA 设备暴露给脚本层几何工作负载。**多 GPU 基准** (multi-GPU benchmark) 使

用弱扩展 (weak scaling): 每张 GPU 接收相同数量的查询点, 总查询数随 GPU 数量增长。每个 worker 拥有独立 FTCL interpreter, 在不同 CUDA device 上创建 UVec inputs, 并分别在 `cuda:0`、`cuda:1`、...、`cuda:7` 上启动相同 geometry command。报告的 speedup 是相对于 1 GPU 的 throughput speedup。

基准运行时需要让所有设备可见, 例如:

```
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 \
FTCL_MULTI_GPU_MODE=paper \
FTCL_MULTI_GPU_SCALING=weak \
test/bench_geometry_multi_gpu
```

记录运行中, $N = 32768$, 每张 GPU 接收 $Q = 4096$ 个查询对象, 每行报告三轮中位数。因此 1、2、4、8 张 GPU 对应的总查询数分别为 4096、8192、16384 与 32768。计时器测量从同时释放所有 worker 线程到所有 worker 获得输出 UVec 句柄的 wall-clock 区间。长度检查与输出销毁在计时区间之后执行, 因此计时聚焦命令分发、UVec 句柄查找、输出分配、CUDA 启动、后端同步与结果句柄创建。

该实验还暴露了一个重要的运行时瓶颈。原始 UVec 命令管理器使用全局 mutex 保护整个句柄表, 并在持锁状态下执行几何命令, 导致彼此独立的 CUDA 命令在脚本运行时层被串行化。实现后来改为: 全局 mutex 只保护句柄表查找, 每个 UVec 句柄再拥有独立条目 mutex, 使不同 GPU 上的独立 UVec 句柄可以并发执行。

表 5.3: 多 GPU 几何命令 weak-scaling speedup

算法	1 GPU	2 GPU	4 GPU	8 GPU
距离矩阵	1.00x	0.96x	0.97x	1.12x
最近点	1.00x	2.00x	3.99x	3.74x
圆形范围计数	1.00x	1.98x	3.97x	7.90x

多 GPU 几何扩展性 (Multi-GPU scaling)

弱扩展实验：每张 GPU 保持固定工作量，查询被分配到 CUDA 0 到 7 号设备。
相对 1 GPU 的加速比 (x)

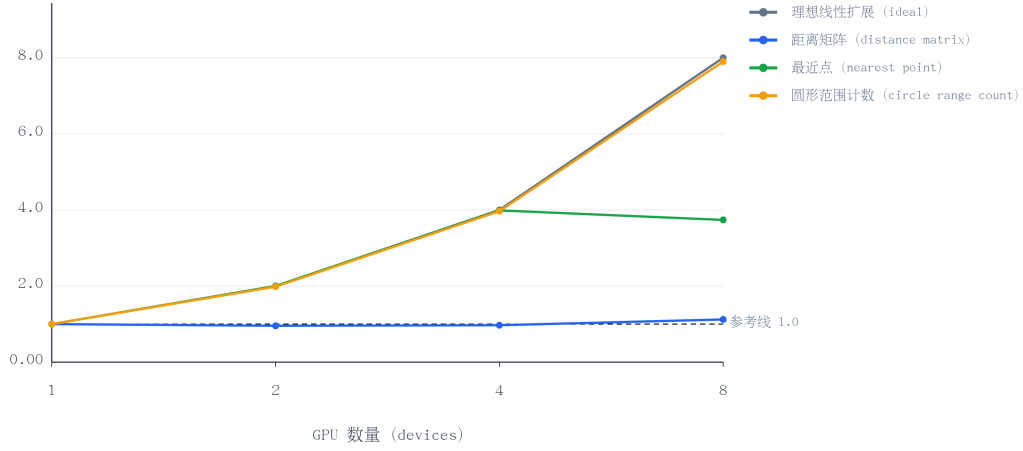


图 5.9: 1、2、4、8 张 A800 GPU 的 weak-scaling speedup

5.8.1 多 GPU 弱扩展模型 (Weak-scaling Model)

为了解释表 5.3 中三类算法差异较大的扩展结果，需要先明确本实验采用的是弱扩展 (weak scaling)，而不是强扩展 (strong scaling)。强扩展固定总工作量，观察更多 GPU 是否能更快完成同一份任务；弱扩展则固定每张 GPU 的局部工作量，让总工作量随 GPU 数量增加。本文的 multi-GPU benchmark 属于后一种情形：每张 GPU 都处理规模相同的一份 geometry workload。

设 g 为 GPU 数量， W 为单张 GPU 上的局部工作项数， T_1 为 1 张 GPU 处理 1 份任务的 wall-clock time， T_g 为 g 张 GPU 同时处理 g 份任务的 wall-clock time。则 1 GPU 与 g GPU 的吞吐量分别为

$$\text{Throughput}(1) = \frac{W}{T_1}, \quad \text{Throughput}(g) = \frac{gW}{T_g}.$$

因此 weak-scaling throughput speedup 定义为

$$S_{\text{weak}}(g) = \frac{\text{Throughput}(g)}{\text{Throughput}(1)} = g \frac{T_1}{T_g}.$$

这个公式给出了一个直接判据：如果 $T_g \approx T_1$ ，则 $S_{\text{weak}}(g) \approx g$ ，说明系统接近线性扩展；如果 $T_g \approx 2T_1$ ，则 8 GPU 时 $S_{\text{weak}}(8) \approx 4$ ；如果 $T_g \approx 8T_1$ ，则 $S_{\text{weak}}(8) \approx 1$ ，说明增加 GPU 主要增加了总工作量，却没有提高整体吞吐效率。

更真实的多 GPU wall-clock time 不能只写成 CUDA kernel time，而应拆分为

$$T_g = \max_{1 \leq i \leq g} T_{\text{local},i} + T_{\text{shared}}(g) + T_{\text{serial}}(g).$$

其中, $T_{\text{local},i}$ 表示第 i 张 GPU 的本地执行时间, 包括 kernel computation、本地 output write 和 device-side allocation 等; $T_{\text{shared}}(g)$ 表示随 GPU 数量增加而出现的共享资源竞争, 例如 CUDA runtime、host allocator、UVec manager、NUMA/PCIe/driver contention; $T_{\text{serial}}(g)$ 表示真正串行或近似串行的部分, 例如全局锁、统一调度、结果句柄管理、join 和 handle-table 操作。理想情况下,

$$T_{\text{shared}}(g) \approx 0, \quad T_{\text{serial}}(g) \approx 0,$$

于是 $T_g \approx T_1$, 从而 $S_{\text{weak}}(g) \approx g$ 。反之, 如果共享路径或串行路径被大输出放大, 使得 $T_g \approx gT_1$, 那么

$$S_{\text{weak}}(g) = g \frac{T_1}{gT_1} = 1.$$

这正是距离矩阵在 8 GPU 下接近 1 倍 throughput speedup 的数学含义。

三个算法的核心差异不仅是计算复杂度, 更是输出复杂度。它们的主要计算量都可以写成 $O(NQ)$, 但输出规模不同, 如表 5.4 所示。

表 5.4: 多 GPU 扩展中的计算复杂度与输出复杂度

算法	计算量	输出量	多 GPU 表现解释
圆形范围计数	$O(NQ)$	$O(Q)$	输出紧凑, 共享瓶颈弱, 接近线性扩展
最近点	$O(NQ)$	$O(Q)$	输出紧凑, 但 8 GPU 时调度与同步开销上升
距离矩阵	$O(NQ)$	$O(NQ)$	完整物化大矩阵, 输出路径成为主瓶颈

圆形范围计数中, 每个 query 只输出一个 count, 因此 $O_{\text{range}} = O(Q)$ 。输出很小, UVec output allocation、global memory write 和 host-side result management 都相对轻量, 所以在 8 GPU 时 $T_8 \approx T_1$, 实验结果达到 $7.90\times$, 接近理想线性扩展。

最近点查询同样每个 query 只输出一个最近点结果, 输出规模为 $O_{\text{nearest}} = O(Q)$ 。因此 1、2、4 GPU 时扩展较好; 但到 8 GPU 时, worker scheduling、CUDA synchronization、output allocation 与 runtime contention 开始变得明显。由表 5.3 的 $S_{\text{weak}}(8) = 3.74$ 可反推

$$T_8 = 8 \frac{T_1}{S_{\text{weak}}(8)} \approx 2.14T_1,$$

因此吞吐加速约为 4 倍，而不是 8 倍。

距离矩阵的情况不同。它不仅计算所有 point pairs，还要完整输出所有距离，因此输出规模为 $O_{\text{dist}} = O(NQ)$ 。在本文 multi-GPU 实验设置中，每张 GPU 需要输出

$$32768 \times 4096 = 134217728$$

个距离值。如果每个距离使用 4 字节 float 表示，则单张 GPU 的输出约为

$$134217728 \times 4 \approx 512 \text{ MiB},$$

8 张 GPU 的总输出约为 4 GiB。因此距离矩阵的问题不是缺少并行计算，而是当前实现会完整物化一个巨大输出矩阵，并放大 output UVec allocation、global memory write、CUDA runtime synchronization、handle management 和 result object lifetime management 等成本。由 $S_{\text{weak}}(8) = 1.12$ 可反推

$$T_8 = 8 \frac{T_1}{1.12} \approx 7.14 T_1,$$

因此 throughput speedup 接近 1 倍。

为了用一个更紧凑的量描述这种现象，可引入**有效瓶颈比例**（effective bottleneck ratio） ρ ，并用简化模型

$$T_g = T_1 ((1 - \rho) + \rho g)$$

近似多 GPU wall-clock time。对应的 weak-scaling speedup 为

$$S_{\text{weak}}(g) = \frac{g}{(1 - \rho) + \rho g} = \frac{g}{1 + \rho(g - 1)}.$$

这里的 ρ 不是严格意义上的 serial code fraction，而是在该实验设置下没有随 GPU 数量并行扩展、反而表现为共享或串行瓶颈的有效比例。由 $g = 8$ 的实测 speedup 反推

$$\rho = \frac{g/S_{\text{weak}}(g) - 1}{g - 1},$$

可得到表 5.5。

表 5.5: 由 8 GPU weak-scaling 结果反推的有效瓶颈比例

算法	$S_{\text{weak}}(8)$	ρ
圆形范围计数	$7.90\times$	0.18%
最近点	$3.74\times$	16.3%
距离矩阵	$1.12\times$	87.8%

这个模型解释了表 5.3 的主要现象：圆形范围计数几乎没有有效瓶颈，所以接近 8 倍；最近点存在中等运行时瓶颈，所以约 4 倍；距离矩阵大部分时间被输出路径与共享瓶颈主导，所以接近 1 倍。换言之，多 GPU 扩展不仅取决于计算复杂度，还取决于输出复杂度、数据驻留策略、运行时锁粒度与同步路径。

直接结论：圆形范围计数接近线性多 GPU 吞吐量扩展，而距离矩阵暴露出输出主导型瓶颈。

结果说明多 GPU 执行有价值，但强烈依赖算法形态。圆形范围计数几乎线性扩展到 8 张 GPU，因为每个查询只产生一个紧凑结果，且工作模式规则。最近点扩展到 4 张 GPU 表现良好，但到 8 张 GPU 后下降，说明运行时开销、分配与同步开始占据主导。距离矩阵的多 GPU 扩展最弱，因为每个查询会产生大规模输出；输出分配与全局内存流量主导了命令时间。这是一个有价值的负面结果：系统已可使用 8 张 GPU，但要让大规模输出 kernel 真正扩展，还需要 memory pool、持久输出缓冲区与 CUDA stream。

本文已经通过端到端的实验，观察到这个瓶颈现象。但是没有进一步对各个子阶段进行测试和分析，因此不对具体耗时的来源作过度归因，保证严谨性。

5.9 盈亏平衡分析 (Break-even Analysis)

CUDA 加速存在**固定开销** (fixed overhead)。小规模工作负载可能在 CUDA 上更慢，因为 kernel 启动、同步、分配与命令分发会压过真正的计算量。图 5.10 以工作项数 $N \times Q$ 为横轴、加速比为纵轴。

CUDA 盈亏平衡点 (break-even point)

按工作项数量 $N \times Q$ 密集扫描；加速比超过 1.0 表示 CUDA 快于 CPU。
加速比 (CPU 时间 / CUDA 时间)

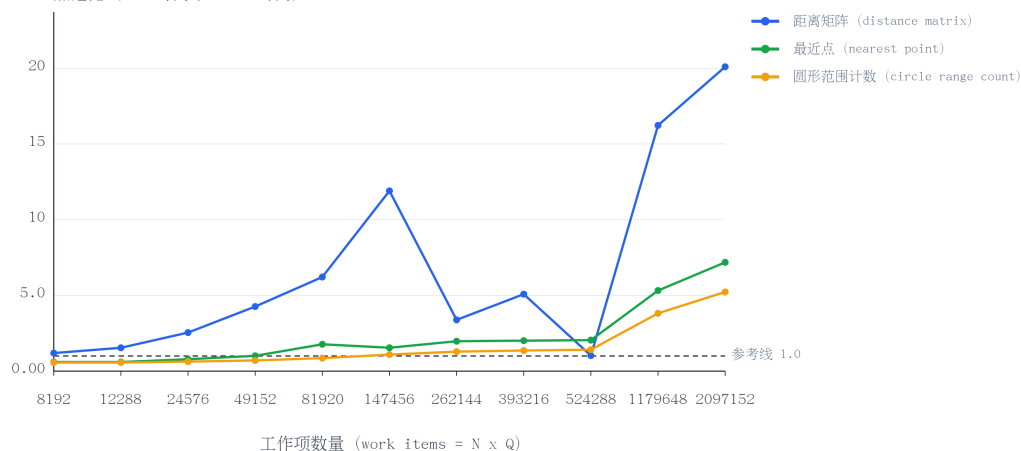


图 5.10: 按工作项统计的 CUDA 盈亏平衡分析

直接结论：只有工作项数量足以摊销启动、分配、同步与 UVec 管理成本后，CUDA 才真正值得使用。

保存的数据估计距离矩阵的盈亏平衡点约为 8192 个工作项，最近点约为 47848，圆形范围计数约为 121209。这些值并非普适常数，而依赖硬件、驱动状态、输入规模、输出规模，以及测量是否包含读回。

5.10 并发几何 (Concurrent Geometry)

并发几何实验如图 5.11 与图 5.12 所示。CPU 吞吐量随 worker 数量增加而提升，直至出现竞争；CUDA 吞吐量更平缓，因为多个脚本 worker 会竞争同一张 GPU 与同步点。

并发几何吞吐量 (concurrent throughput)

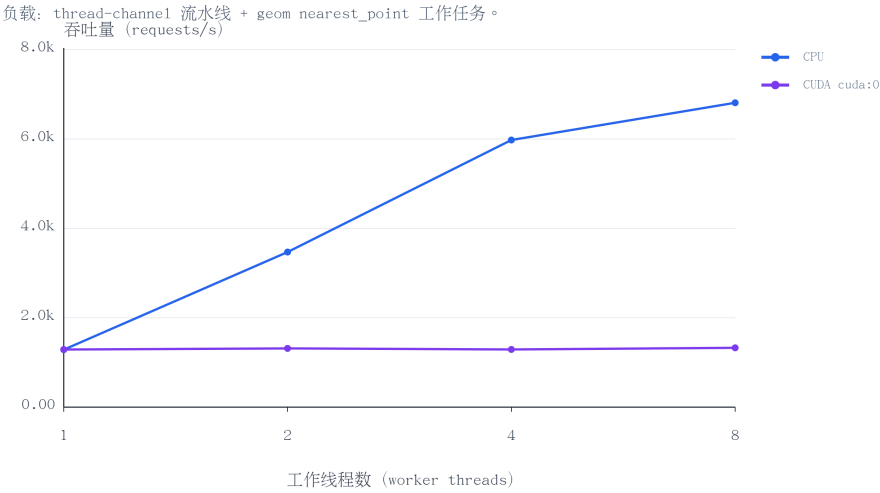


图 5.11: 并发几何工作负载的吞吐量

直接结论: 增加脚本 worker 数量只能在共享运行时或 GPU 资源成为瓶颈之前提升吞吐量。

并发几何尾延迟 (latency tail)

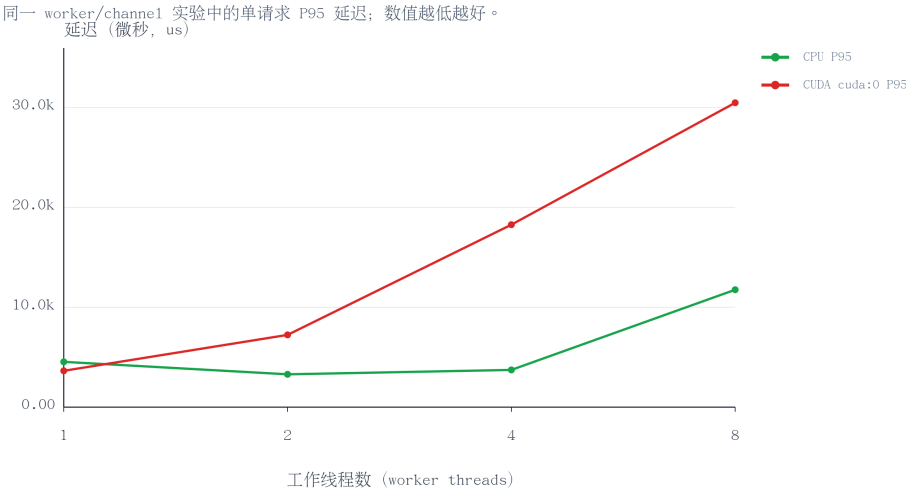


图 5.12: 并发几何工作负载的 P95 延迟

直接结论: 竞争会推高 P95 延迟, 因此后续需要专用调度、内存池与基于 stream 的 CUDA 执行。

5.11 小结

本章从正确性、性能与可扩展性三个维度评估 FTCL。语义回归与 CPU/GPU 等价性测试证明系统具有可验证的正确性基础；单 GPU 与多 GPU 实验显示 CUDA 后端在大量独立工作项上具有明显优势；并发实验则指出真正的瓶颈经常来自分配、读回、加锁与调度。

评估有意采用端到端口径。主要结论是：数据驻留、输出规模与运行时同步，往往决定脚本层异构系统的实际性能。

第 6 章 应用与讨论

6.1 从 STA 的 Tcl 范式到 FTCL 空间计算

在电子设计自动化（Electronic Design Automation, EDA）领域，静态时序分析（Static Timing Analysis, STA）工具长期使用 Tcl 作为交互与自动化脚本语言。这里的 Tcl 并不负责直接完成大规模图遍历、时序传播或约束求解；真正的重计算通常发生在 C/C++ 实现的时序图、设计数据库和分析引擎中。Tcl 的作用是提供一层稳定、可组合、可自动化的**控制界面**（control interface）：用户用脚本读取设计、设置时钟、声明约束、调用分析、筛选路径并生成报告^[1,2]。

这种模式可以概括为：

脚本意图 → 原生命令 → 常驻数据 → 高性能后端 → 报告或句柄

STA 工具中的 Tcl 命令通常不是孤立函数，而是访问一个持续存在的设计数据库（design database）。例如 `create_clock` 修改约束集合，`report_timing` 触发时序路径查询，`set_false_path` 改变分析规则。用户看到的是脚本命令，系统内部维护的是时序图、库单元、网表、约束、缓存和报告对象。

FTCL 希望在计算几何与海洋空间计算中采用类似思想。脚本层不直接管理 CUDA 指针，也不把大规模点集反复序列化成字符串；它只描述“我要计算什么”。`geom` 命令把脚本请求映射到底层几何算法，UVec 负责让点集、查询集和结果集在 CPU/CUDA 之间保持一致，CUDA kernel 或 CPU 后端负责真正执行批量计算。换言之，FTCL 不是把 Tcl 风格语言当作玩具解释器，而是把它作为**异构空间计算控制层**（heterogeneous spatial-computing control layer）。

HeteroSTA 这类 CPU/GPU 混合 STA 系统说明了一个重要事实：异构加速不是单个 kernel 的问题，而是 workflow、数据管理、调度、内存驻留和报告接口共同作用的系统问题^[20]。FTCL 的问题规模比工业 STA 小，但系统思想相同：如果没有脚本层，底层 kernel 很难被灵活组织；如果没有 UVec，脚本层又会被数据搬运和设备指针拖垮。

表 6.1: STA Tcl 范式与 FTCL 空间计算范式的对应关系

维度	STA 工具中的 Tcl	FTCL 中的空间计算
脚本角色	描述设计输入、时钟约束、例外路径和报告请求	描述点集、覆盖区域、碰撞对象、距离查询和设备选择
常驻数据	网表、时序图、库单元、约束数据库	UVec 点集、查询集、包围盒、距离矩阵和结果句柄
重计算部分	时序传播、路径搜索、约束检查、报告生成	最近点、范围计数、距离矩阵、AABB 碰撞、批量几何 kernel
命令价值	把复杂分析过程拆成可复用脚本流程	把空间计算任务拆成可组合、可复现实验流水线
后端演化	从 CPU STA 到 CPU/GPU 混合 STA	从 CPU 几何后端到 CUDA 与多 GPU 后端

这个类比强化了第 6 章的应用定位：海洋牧场、智慧港口、无人机巡检和船舶避碰并不要求 FTCL 变成完整业务系统。FTCL 更像 STA 工具中的 Tcl 控制台：它提供脚本化入口，把领域对象转化为可计算的数据结构，再把大规模批处理任务交给底层执行引擎。

6.2 FTCL 的空间计算 workflow

从应用角度看，FTCL 的空间计算流程可以分为四步。

第一步是对象几何化。海洋牧场中的浮标、漂浮物、无人机、船舶、网箱和雷达覆盖范围首先被转化为几何对象。点表示位置，圆表示覆盖半径，轴对齐包围盒（Axis-Aligned Bounding Box, AABB）表示粗粒度障碍或船舶外形，距离矩阵表示大规模目标关系。

第二步是脚本编排。用户通过 FTCL 命令组织任务，而不是修改 C++ 或 CUDA 代码。脚本可以设置阈值、选择设备、组合多条查询、保存中间结果并决定是否读回。

第三步是数据驻留。一旦点集进入 UVec，脚本后续传递的是句柄而不是完整数组文本。这样可以避免每个命令都把大数组重新解析、复制和格式化。对于连续的几何流水线，这一点比单个 kernel 是否快更重要。

第四步是后端执行。同一条 geom 命令可以选择 CPU 或 CUDA。小规模任务可以留在 CPU；大规模、独立工作项密集的任务可以放到 CUDA；多个相互独立的任务还可以被分配到多张 GPU 上。

Listing 6.1: FTCL spatial-computing control script

```

set sensors [geom uvec_points $buoys cuda:0]
set targets [geom uvec_points $floating_objects cuda:0]
set uav [geom uvec_points {{$ux $uy}} cuda:0]

set nearest [geom nearest_point $targets $uav cuda:0]
set counts [geom range_count_circle $targets $sonar_regions 120 cuda:0]
set risk [geom collision_aabb $ship_boxes $restricted_zones cuda:0]
set matrix [geom batch_distance_matrix $sensors $targets cuda:0]

```

这段脚本的重点不在于语法复杂，而在于边界清晰：脚本表达任务，geom 解释任务，UVec 保持数据驻留，CPU/CUDA 后端完成计算。它与 STA 脚本调用 report_timing 的思想相似：命令很短，但背后连接的是持续存在的数据结构和高性能分析引擎。

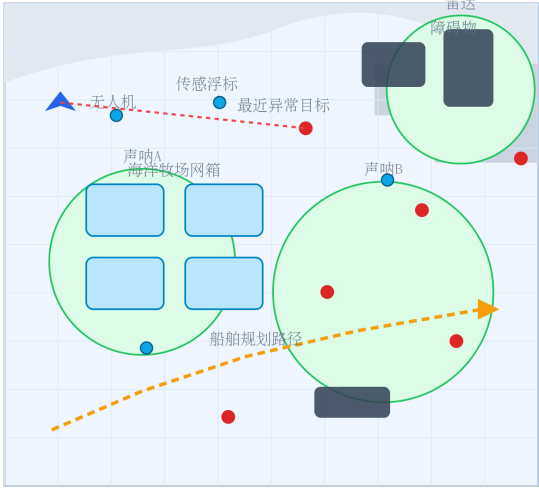
6.3 面向海洋的空间计算演示 (Spatial Computing Demo)

图 6.1 展示了一个面向海洋的空间计算演示场景。它不再使用普通游戏场景，而是把系统放到更贴近海洋牧场、智慧港口、船舶导航和无人巡检的应用语境中。浮标传感器、无人机、漂浮物、船舶、养殖网箱、港口障碍以及声呐/雷达覆盖区域都被表示为几何数据。

在这一场景中，点表示传感器位置、漂浮物、目标观测与无人机位置；圆表示雷达、声呐或相机覆盖范围；AABB 表示船舶、养殖网箱、港区限制区域或矩形障碍；批量距离矩阵则表示传感器与异常目标之间的大规模关系。借助 UVec，这些对象可以跨多个命令驻留在 CUDA 设备上，而不必每一步都转换回脚本文本。

FTCL 海洋空间计算演示 (ocean spatial computing demo)

海洋牧场感知 + 智慧港口安全 + 无人机巡检被组织到同一条脚本流水线中。



FTCL 脚本片段 (script excerpt)

```
set S [geom uvec_points $buoys cuda:0]
set T [geom uvec_points $objects cuda:0]
set U [geom uvec_points {{$ux $uy}} cuda:0]
set near [geom nearest_point $T $U cuda:0]
set cover [geom range_count_circle $T $sonar cuda:0]
set risk [geom collision_aabb $ship $zone cuda:0]
set D [geom batch_distance_matrix $S $T cuda:0]
```

输出语义 (Output semantics) :

near -> 异常目标句柄
cover -> 每个声呐的目标计数
risk -> 船舶/障碍物碰撞标记
dist -> 大规模传感器-目标距离矩阵

图 6.1: FTCL 控制的海洋空间计算场景 (Ocean-oriented spatial computing scene)

图中的脚本流水线使用的仍是第 5 章中的几何命令，但数据语义发生了变化。`geom nearest_point` 可以寻找距离无人机或巡逻船最近的异常漂浮物；`geom range_count_circle` 可以统计雷达、声呐或相机覆盖区域内的目标数量；`geom collision_aabb` 可以检查船舶包围盒是否与礁石、网箱、码头或限制区域重叠；`geom batch_distance_matrix` 可以计算大规模传感器-目标距离关系，为后续分配或告警评分提供基础。

6.4 海洋牧场空间感知

在海洋牧场中，浮标和水下传感器会持续观测水质、网箱状态、漂浮垃圾和异常水面目标。一个简单的 FTCL 脚本可以把传感器位置与目标检测结果载入 UVec 句柄，对每个声呐覆盖圆执行范围查询，并报告离每个巡检点最近的异常目标。

这一场景适合 FTCL，是因为输入天然形成大规模点集。许多查询彼此独立：每个传感器覆盖区域可以单独处理，每个目标到传感器的距离也可以并行计算。当检测目标或传感器数量变大时，CUDA 后端可以承担批量计算，而脚本层仍然保留组合任务的便利性。

更具体地说，海洋牧场的空间感知任务通常包含三类计算。第一类是覆盖统计，即每个声呐或摄像头覆盖范围内有多少目标；第二类是最近异常目标定位，即从某个巡检点或无人机位置出发，寻找最近漂浮物；第三类是全局关系构造，即计算传

传感器集合与目标集合之间的距离矩阵。三类任务都可以被拆成大量独立工作项，因此与 CUDA 的并行执行模型匹配。

FTCL 在这里提供的不是传感器识别模型，而是识别结果之后的空间计算层。传感器模型可以来自外部系统，FTCL 只需要接收目标坐标、覆盖半径和区域边界，再用脚本把它们组织为可复现的分析流程。

6.5 智慧港口空间计算

智慧港口包含泊位、吊机、集装箱、船舶包围盒、航道和限制区域。这些对象可以用点、线段、多边形和包围盒表示。FTCL 可以用 `geom collision_aabb` 检查船舶与港口障碍物的粗粒度碰撞，用 `geom range_count_rect` 统计管理区域内对象数量，用 `geom nearest_point` 查找最近资源或警戒目标。

这里的价值并不是让 FTCL 替代完整港口信息系统，而是提供一个轻量级空间计算层（spatial-computing layer）。使用者可以先用脚本快速原型化港口空间规则，再把大规模批量运算移交给 CUDA 后端，而不必改变高层命令结构。

智慧港口的规则往往频繁变化。例如安全距离阈值、限制区域边界、告警等级、船舶类型过滤条件都可能随管理策略调整。如果每次修改都需要重新编译底层程序，实验和迭代成本会很高。FTCL 的脚本层可以把这类规则变化留在文本层，把稳定的几何计算保留在原生后端。这个分工与 STA 工具非常相似：约束脚本经常变化，但底层时序引擎保持稳定。

6.6 无人机水面漂浮物巡检

无人机常用于近岸水面巡检。无人机轨迹可以存储为点序列，检测到的漂浮物可以存储为另一个点集，相机覆盖范围可以表示为圆形查询。FTCL 可以计算最近异常目标、统计每一帧覆盖区域内的检测数量，并构造采样点与可疑目标之间的距离矩阵。

这个例子说明了**脚本层异构计算**（script-level heterogeneous computing）的意义。高层巡检逻辑经常变化：阈值、目标类别和覆盖半径会随任务调整；而底层几何 kernel 相对稳定，可以由 GPU 加速承担。

无人机巡检还体现了 FTCL 对实验复现的价值。同一批检测点可以在不同覆盖半径、不同告警阈值、不同设备后端下重复执行，生成可比较的结果。研究者可以

把脚本、输入数据、CSV 输出和图表生成过程一起保存，从而形成完整实验链条。这比只提供一个 C++ 几何函数更接近毕业论文或工程实验需要的形态。

6.7 船舶路径规划与碰撞检测

船舶导航需要把规划路径与静态、动态障碍物进行检查。在当前 FTCL 几何子集中，粗粒度碰撞检测可以由 AABB 命令完成，线段相交与距离命令则可以支撑后续路径级检查。船舶路线也可以与养殖网箱、礁石、港口边界和移动障碍包围盒进行风险评估。

这一场景指向自然的扩展方向。在当前批量几何命令之后，系统可以继续加入空间网格、折线距离、路径风险评分和动态障碍预测。这些扩展会让 FTCL 更直接服务于海洋与港口空间计算任务。

路径规划本身可以很复杂，但 FTCL 可以先承担其中可明确批处理的部分。例如对候选路径采样点与障碍物集合计算最小距离，对多个候选路径统计碰撞次数，对不同安全半径重复评估风险。这样的任务具有明显的数据并行结构，适合从 CPU 后端迁移到 CUDA 后端。

6.8 算法可视化

图 6.2 展示了几个代表性算法：点在多边形内判定、最近点、圆形范围查询和 AABB 碰撞。这样的图对答辩非常有价值，因为它让评阅者不看代码也能理解工作负载本身。

几何算法可视化 (algorithm visualization)

展示 FTCL 脚本接口中四类代表性 geom 命令。

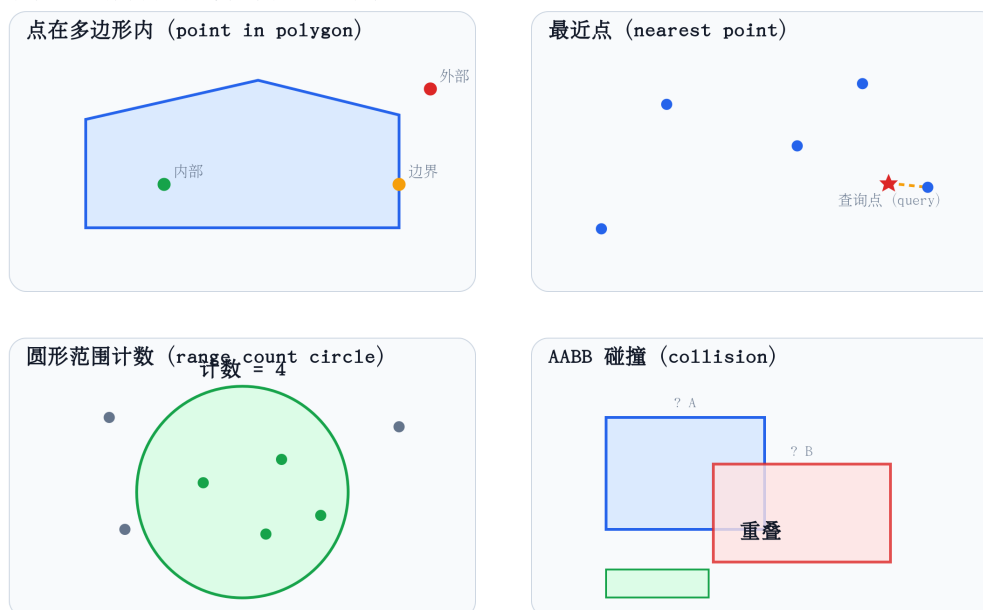


图 6.2: 代表性几何算法可视化 (Representative geometry algorithms)

这些算法不是随意拼接的功能列表，而是共同构成空间计算的基本原语。最近点回答“离我最近的目标是谁”；范围计数回答“某个覆盖区域内有多少对象”；碰撞检测回答“规划对象是否进入危险区域”；距离矩阵回答“两个对象集合之间的全局关系是什么”。当这些问题被脚本组织起来时，FTCL 就具备了类似 STA Tcl flow 的工作流表达能力。

6.9 与普通几何库的区别

普通几何库通常提供 C++ 函数，例如 `distance()` 或 `intersect()`。FTCL 与它们的区别主要有四点。

- 它通过脚本暴露几何操作，使用者可以组合任务而不必重新编译 C++。
- 它使用 UVec 句柄，避免大数组在脚本文本与原生缓冲区之间反复转换。
- 它在同一命令接口下同时支持 CPU 后端与 CUDA 后端。
- 它包含测试、随机差分检查、等价性检查与生成图表，能够支撑论文级评估。

因此，本项目不是单纯的几何函数集合。它的价值在于解释器设计、数据管理、异构计算与实验验证的整合。

从应用定位看，这也改变了项目的叙事方式。FTCL 不只是计算几何脚本系统，也可以被描述为面向海洋与港口空间分析的原型异构计算底座。几何命令提供第一层算法能力，UVec 与 CUDA 提供执行底座，脚本语言提供控制层。

更准确地说，FTCL 的定位接近“领域脚本控制层 + 原生计算核心”。这正是许多工程软件采用 Tcl 的原因：脚本层负责开放、灵活和可组合；原生层负责性能、内存和系统边界。FTCL 把这个经典思想迁移到计算几何与异构空间计算中。

6.10 局限性

当前系统仍有若干局限。

第一，UVec 尚未完全对齐 `std::vector`。它已经支持跨设备存储与脚本句柄，但还应补充迭代器、容量管理、插入、删除和更强的异常安全保证。

第二，CUDA 后端仍然比较直接。它尚未使用内存池、异步流、高级归约、共享内存分块或空间索引。这些技术对降低开销和提高性能很重要。

第三，词法流解析器在架构上更清晰，但不总是比传统解析器更快。未来应减少 token 拷贝，压缩空白处理，避免对子脚本重复词法分析，并引入 AST 或字节码执行。

第四，基准图表应在固定硬件环境下进行更多轮重复，并补充置信区间。部分 CUDA 盈亏平衡点会波动，因为端到端计时包含分配、同步与系统调度噪声。

第五，当前海洋与港口案例仍属于原型级空间计算。它能够证明 FTCL 可以表达和执行相关几何任务，但尚未接入真实海洋传感器数据流、港口业务数据库或船舶自动识别系统（Automatic Identification System, AIS）。这些集成属于后续工程化方向。

6.11 小结

本章把 FTCL 从“几何脚本系统”提升到“海洋空间计算原型平台”的叙事层面。通过类比 STA 使用 Tcl 组织复杂分析流程，本文说明 FTCL 也可以作为计算几何与海洋空间任务的控制层：脚本表达意图，UVec 保持数据驻留，CPU/CUDA 后端提供执行能力。

应用层并不改变核心算法，而是改变这些算法的解释方式。当点、圆、包围盒和距离矩阵与海洋导向的空间决策联系起来时，FTCL 的论证会更有说服力。

In short, FTCL follows the same systems principle as Tcl-driven engineering tools: keep the user-facing language small, keep the data resident, and move heavy computation into optimized native backends.

结论与展望

本文完成了一个从 Tcl 风格解释器出发、并逐步扩展为 CPU/CUDA 混合计算几何脚本平台的端到端系统。FTCL 目前已支持实用的语言子集、线程通道、UVec 跨设备向量、CPU 几何算法、CUDA kernel、测试框架、基准流水线与论文图表生成流程。实验结果表明，当工作项足够多且彼此独立时，GPU 后端能够带来明显收益；但当输入规模过小、输出过大或需要立即读回时，并行执行的优势会被启动、分配、同步与数据搬运成本抵消。

后续工作可从四个方向继续推进。第一，UVec 应进一步向标准向量接口对齐，包括迭代器、容量管理、插入/删除以及更强的异常安全保证。第二，CUDA 后端应加入内存池、异步 stream 与更好的批处理，以降低分配与同步成本。第三，解析路径可以继续向 token-stream 解析、AST 表示、字节码执行与表达式级 Pratt 解析演进。第四，几何库可以扩展空间索引、动态最近邻搜索、多边形布尔运算，并构建更贴近海洋牧场、智慧港口、无人机巡检与船舶避碰的应用演示。

FTCL 的价值不只是“能运行几何命令”，而是把脚本层表达、跨设备内存、并发执行与可复现实验放进同一个系统边界中。未来最重要的方向，是将 FTCL 从原型平台发展为可复用的异构空间计算运行时。

致 谢

感谢党和国家对我的关怀，感谢人民和社会对我的培养与支持。

感谢我的指导老师纪兆辉老师在论文写作过程中给予的悉心指导。

感谢汪前进老师、朱敏老师、胡文彬老师、李慧老师、贾冬宝老师、吴加莹老师、韩璧如老师等各位老师在本科阶段对我的教导与帮助。

感谢冷严伟、刘好、孙京权等一路同行的伙伴。愿我们在未来继续坚定理想、脚踏实地，成就更有意义的事业。感谢班长吕勇同学在班级事务中的辛勤付出与帮助。

感谢我的家人长期以来的理解、支持与陪伴。

周恩来总理少年时曾立下“为中华之崛起而读书”的志向。今后，我也将以此自勉，继续锤炼专业本领，坚守初心，努力以所学所长服务国家发展、回报人民和社会。

参考文献

- [1] John K. Ousterhout. *Tcl and the Tk Toolkit*. Addison-Wesley, 1994.
- [2] Tcl Core Team. *Tcl Documentation*. 2026. url: <https://www.tcl.tk/doc/> (visited on 05/28/2026).
- [3] Mark de Berg et al. *Computational Geometry: Algorithms and Applications*. 3rd ed. Springer, 2008.
- [4] Franco P. Preparata and Michael Ian Shamos. *Computational Geometry: An Introduction*. Springer, 1985.
- [5] NVIDIA. *CUDA C++ Programming Guide*. 2026. url: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (visited on 05/28/2026).
- [6] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley, 2006.
- [7] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [8] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. url: <https://craftinginterpreters.com/>.
- [9] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.
- [10] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3rd ed. Addison-Wesley, 1997.
- [11] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd ed. Addison-Wesley, 1998.
- [12] Hanan Samet. *Foundations of Multidimensional and Metric Data Structures*. Morgan Kaufmann, 2006.
- [13] Erik Lindholm et al. “NVIDIA Tesla: A Unified Graphics and Computing Architecture”. In: *IEEE Micro* 28.2 (2008), pp. 39–55. doi: [10.1109/MM.2008.31](https://doi.org/10.1109/MM.2008.31).
- [14] Michael Garland et al. “Parallel Computing Experiences with CUDA”. In: *IEEE Micro* 28.4 (2008), pp. 13–27. doi: [10.1109/MM.2008.57](https://doi.org/10.1109/MM.2008.57).

- [15] Shane Ryoo et al. “Optimization Principles and Application Performance Evaluation of a Multithreaded GPU Using CUDA”. In: *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 2008, pp. 73–82. doi: [10.1145/1345206.1345220](https://doi.org/10.1145/1345206.1345220).
- [16] Xinxin Mei and Xiaowen Chu. “Dissecting GPU Memory Hierarchy Through Microbenchmarking”. In: *IEEE Transactions on Parallel and Distributed Systems* 28.1 (2017), pp. 72–86. doi: [10.1109/TPDS.2016.2549523](https://doi.org/10.1109/TPDS.2016.2549523).
- [17] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2019.
- [18] John E. Stone, David Gohara, and Guochun Shi. “OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems”. In: *Computing in Science and Engineering* 12.3 (2010), pp. 66–73. doi: [10.1109/MCSE.2010.69](https://doi.org/10.1109/MCSE.2010.69).
- [19] Michael J. Flynn. “Some Computer Organizations and Their Effectiveness”. In: *IEEE Transactions on Computers* C-21.9 (1972), pp. 948–960. doi: [10.1109/TC.1972.5009071](https://doi.org/10.1109/TC.1972.5009071).
- [20] Zizheng Guo et al. “HeteroSTA: A CPU-GPU Heterogeneous Static Timing Analysis Engine with Holistic Industrial Design Support”. In: *Proceedings of the 31st Asia and South Pacific Design Automation Conference*. 2026, pp. 1–7. doi: [10.1109/ASP-DAC66049.2026.11420338](https://doi.org/10.1109/ASP-DAC66049.2026.11420338).
- [21] Kitware. *CMake Documentation*. 2026. url: <https://cmake.org/documentation/> (visited on 05/28/2026).
- [22] Brent B. Welch, Ken Jones, and Jeffrey Hobbs. *Practical Programming in Tcl and Tk*. 4th ed. Prentice Hall, 2003.
- [23] David B. Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. 3rd ed. Morgan Kaufmann, 2016.
- [24] Jason Sanders and Edward Kandrot. *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley, 2010.
- [25] John D. Owens et al. “GPU Computing”. In: *Proceedings of the IEEE* 96.5 (2008), pp. 879–899. doi: [10.1109/JPROC.2008.917757](https://doi.org/10.1109/JPROC.2008.917757).

- [26] John Nickolls et al. “Scalable Parallel Programming with CUDA”. In: *Queue* 6.2 (2008), pp. 40–53. doi: [10.1145/1365490.1365500](https://doi.org/10.1145/1365490.1365500).

构建、测试与复现命令

本项目已开源，完整代码、测试用例与图表生成脚本见 GitHub 仓库：

<https://github.com/homer-fang/ftcl>

克隆仓库后，可按下列命令完成构建、测试与实验复现。以下路径以 WSL 下的 /mnt/d/ftcl/ftcl 为例；若从 GitHub 克隆，请将工作目录替换为本地克隆路径。

.1 CPU 构建与测试

```
git clone https://github.com/homer-fang/ftcl.git
cd ftcl
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_TESTS=ON \
  -DFTCL_ENABLE_CUDA=OFF \
  -DFTCL_CXX_STANDARD=20
cmake --build build -j8
ctest --test-dir build --output-on-failure
```

.2 CUDA 构建

```
cmake -S . -B /tmp/ftcl-build-geometry-cuda \
  -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_TESTS=ON \
  -DFTCL_ENABLE_CUDA=ON \
  -DFTCL_CXX_STANDARD=20
cmake --build /tmp/ftcl-build-geometry-cuda -j8
```

.3 核心基准测试

```
./build/test/bench_ftcl ./docs/data/benchmarks  
python3 ./docs/scripts/benchmarks/plot_benchmarks.py \  
./docs/data/benchmarks ./docs/figures/benchmarks
```

.4 几何图表生成

```
python3 docs/scripts/geometry/generate_geometry_figures.py \  
--build-dir /tmp/ftcl-build-geometry-cuda  
  
python3 docs/scripts/geometry/generate_geometry_study_addons.py \  
--build-dir /tmp/ftcl-build-geometry-cuda
```

.5 论文编译

```
latexmk -xelatex -interaction=nonstopmode -output-directory=build main.tex
```

按当前 latexmk 配置，生成的 PDF 输出为 build/main.pdf。